

UPPSALA
UNIVERSITET

IT kDV 26 037

Degree project 15 hp

2026

AI-Based Mushroom Identification Using Image Recognition and Trait-Based Classification

A Benchmark Evaluation of Image Recognition, Trait-Based Reasoning, and Multimodal LLM Synthesis

Ian Nyfelt



Bachelor's Programme in Computer Science



UPPSALA
UNIVERSITET

AI-Based Mushroom Identification Using Image Recognition and Trait-Based Classification

Ian Nyfelt

Abstract

Mushroom identification is a safety-critical task where visual similarity between edible and toxic species can make image-only classification unreliable. This thesis investigates whether combining machine learning, trait-based identification, and multimodal large language model reasoning improves mushroom-species identification compared with using each method as a standalone approach.

The implemented system combines an EfficientNet-B3 convolutional neural network fine-tuned using transfer learning, a YOLOv8-based part-segmentation model, a deterministic mushroom trait extractor, an XML-based identification-key traversal engine, a trait-database comparator, and a multimodal LLM synthesis stage. The system was evaluated using Benchmark 5, a held-out benchmark of 30 mushroom specimens divided into standard, confusing lookalike, and out-of-distribution cases. The benchmark compared standalone methods in two configurations: A1, which used automatically extracted traits, and A2, which replaced these traits with oracle traits. These standalone results were then compared with two unified pipeline variants: B1, which synthesised the A1-style extracted-trait evidence, and B2, which synthesised the A2-style oracle-trait evidence. This was done in order to isolate the effect of trait-extraction quality.

The results show that the unified pipeline did not outperform the strongest standalone methods overall. B1 reached 50.0% top-1 accuracy and B2 reached 53.3%, compared with 56.7% for the CNN classifier and 73.3% for the oracle-trait database comparator. However, the unified approach corrected selected CNN errors, indicating that evidence synthesis can be useful in individual cases. The main limitations of this project were unreliable part segmentation, heuristic trait extraction, low key-traversal coverage, and the difficulty of making the LLM consistently use structured evidence.

The thesis contributes an implemented multimodal mushroom-identification prototype and benchmark framework. The results suggest that unified evidence integration is promising, but that current implementation of segmentation, trait extraction, and synthesis design need to be improved before such a system can reliably outperform strong standalone baselines.

Faculty of Science and Technology

Uppsala university, Place of issue Uppsala

Handledare: Per Nyfelt Ämnesgranskare: Didem Gürdür Broo

Examinator: Johannes Borgström

Table of Contents

1 Introduction **4**
1.1 Problem Formulation . . . 4
1.2 Exploratory market conditions and analysis . . . 5
1.3 Research Question . . . 5
1.4 Contributions . . . 5
1.5 Thesis Outline . . . 6

2 Background **7**
2.1 Mushroom Identification as a Knowledge-Intensive Task . . . 7
2.2 Species Knowledge and Identification Keys . . . 7
2.3 Image-Based Recognition . . . 7
2.4 Artificial Intelligence and AI systems . . . 8
2.5 Large Language Models and Multimodal AI . . . 8
2.6 Image Segmentation . . . 9
2.7 Classical image processing methods . . . 9
2.7.1 Clustering and colour quantisation . . . 10
2.7.2 Mask-based region analysis . . . 10
2.7.3 Contour-based shape analysis . . . 10
2.7.4 Edge, line, and blob detection . . . 10
2.7.5 Threshold-based heuristic classification . . . 10
2.8 Combining Multiple Evidence Sources . . . 11

3 Method **12**
3.1 Benchmark Design Overview . . . 12
3.2 Dataset Construction . . . 14
3.3 CNN-Based Image Classification . . . 14
3.4 Image Segmentation with YOLOv8 . . . 15
3.5 Classical Visual Trait Extraction . . . 15
3.5.1 Colour representation and clustering . . . 16
3.5.2 Shape heuristics and threshold calibration . . . 16
3.5.3 Hymenophore classification with competing heuristics . . . 16
3.5.4 Puffball case and the whole-body path . . . 16
3.5.5 Morphology case derivation . . . 16
3.5.6 Output schema: part-aware and legacy fields . . . 17
3.5.7 Confidence scores as quality gates . . . 17
3.6 Trait-Based Identification Key and Database . . . 17
3.7 Benchmark A1 and A2 MLLM . . . 18
3.7.1 A1: LLM capability . . . 18
3.7.2 A2: LLM with Oracle Traits . . . 18
3.8 Unified LLM Synthesis . . . 19
3.9 Prompt Construction and Generation Settings . . . 19
3.9.1 Input Construction . . . 20
3.9.2 Fallback Behaviour . . . 20
3.9.3 Response Parsing and Agreement Evaluation . . . 21
3.10 Evaluation Procedure . . . 21
3.10.1 Benchmark Evaluation Procedure . . . 21

4 Results **23**
4.1 Overall Benchmark Performance . . . 23
4.2 Comparative Accuracy Differences . . . 23
4.3 Scenario-Level Results . . . 24
4.4 Low-Coverage Identification-Key Results . . . 24
4.5 Confusing Lookalike Results . . . 24
4.6 Qualitative Segmentation Errors . . . 25
4.7 Fallback and Agreement Metadata . . . 25
4.8 Example Per-Specimen Output . . . 25
4.9 Complementarity Between CNN and Unified Pipeline Predictions . . . 25

4.10 Summary of Results . . . 26

5 Discussion **27**
5.1 Answering the Research Question . . . 27
5.2 Benchmark Performance . . . 27
5.2.1 Standalone Performance . . . 27
5.2.2 Unified Pipeline Performance . . . 27
5.2.3 Complementary Errors Between CNN and Unified Systems . . . 28
5.3 MLLM Synthesis Limitations . . . 28
5.4 Key Traversal and Interpretability . . . 29
5.5 Trait Extraction as a Bottleneck . . . 29
5.6 Benchmark, Dataset, and Comparability Limitations . . . 30
5.7 Future Work . . . 30

6 Conclusions **33**

References **34**

A Appendix A: Project Artefact Overview **36**
A.1 Benchmark-Relevant Source Code . . . 36
A.2 Data and Knowledge Files . . . 36
A.3 Model Checkpoints and Training Artefacts . . . 37
A.4 Benchmark Outputs . . . 37
A.5 Application Components Outside the Benchmark Scope . . . 37

B Appendix B: Benchmark Model Training and Execution in Google Colab **38**
B.1 Colab Runtime . . . 38
B.2 CNN Species Classifier . . . 38
B.3 YOLOv8 Part-Segmentation Model . . . 38
B.4 Benchmark Use of the Checkpoints . . . 39
B.5 Running the Benchmark in Google Colab . . . 39

C Benchmark Per-Specimen Results **42**

1 Introduction

Mushroom foraging is a widely practiced leisure activity, and wild mushrooms are valued both as a culinary ingredient and as a source of nutrition, as they contain valuable nutrients such as vitamins, minerals, and in some cases vitamin D. However, the ability to reliably distinguish edible species from toxic lookalikes requires considerable expertise that often many foragers lack. The consequences of misidentification can be severe: toxic species affect different physiological systems depending on the compounds involved—the fly agaric (*Amanita muscaria*) targets the nervous system, the yellow stainer (*Agaricus xanthodermus*) causes gastrointestinal distress, the destroying angel (*Amanita phalloides*) causes fatal organ damage, the brown roll-rim (*Paxillus involutus*) can trigger haemolytic reactions, and the common ink cap (*Coprinopsis atramentaria*) induces alcohol sensitivity [1].

The risk is not evenly distributed across the population. In some municipalities, such as Ockelbo, an increase in mushroom poisoning incidents has been observed among individuals who do not have Swedish as their first language, suggesting that language barriers may limit access to relevant safety information [2].

These circumstances motivate the development of accessible, technology-based tools that can assist amateur foragers in accurately identifying mushrooms and distinguishing edible species from their toxic lookalikes.

Mushroom identification is an challenging application area for artificial intelligence because it combines visual recognition, domain knowledge, and high safety requirements. A photograph can capture some initial information however to identify mycological species requires more context such as cap shape, gill attachment, growth pattern, habitat, and season. A single photograph therefore rarely captures all information needed for a reliable judgement of AI. This problem is made more serious by the fact that several edible species have toxic lookalikes. The practical challenge therefore is not only to produce a likely species label, but also to communicate uncertainty and safety warnings in a way that supports responsible use.

Recent work in deep learning has made image-based classification effective in many visual domains [3;4;5]. In contrast, identification keys provide a structured and interpretable way to reason over categorical morphological traits. Although such keys are not learned decision trees, they can be represented as tree-structured decision procedures, where each branch corresponds to a trait choice and each path narrows the set of candidate species. In parallel, transformer-based language models have made it possible to interpret natural-language descriptions and generate structured predictions from text [6;7]. These developments motivate a hybrid synthesis approach to mushroom identification, where image evidence, structured trait reasoning, and language-model synthesis provide complementary sources of evidence. This thesis therefore investigates whether a large language model can improve identification by synthesising multiple evidence sources compared with using each source as a standalone method.

1.1 Problem Formulation

Several studies have applied machine learning to mushroom identification, but the present thesis focuses on a different identification setting.

Lee et al. [8] developed a CNN-based Android application that classifies mushrooms from field photographs into two, three, or five genera, achieving sensitivity and specificity ranging from 89% to 100% across those configurations. Their results show that CNN-based image classification can perform well when the prediction task is framed at genus level and evaluated over a limited number of classes.

The present thesis addresses a related but more fine-grained problem. Instead of predicting genus-level edibility categories, it evaluates species-level identification, where visually similar species may belong to the same genus or share many external traits. This distinction is important for mushroom identification, because safe foraging often depends on fine-grained differences between species rather than only broader genus-level groupings. The results of Lee et al. therefore support the use of CNNs as a relevant image-based baseline, while leaving open the question of whether image-only classification is sufficient for species-level identification.

For this reason, the present thesis investigates whether image-based prediction can be complemented by structured trait information and language-model synthesis. This was done in the form of a benchmark where the evaluated evidence sources include an image-based CNN classifier, segmentation-assisted visual trait extraction, structured trait reasoning through an identification key and trait database, and an LLM-based synthesis component.

The central research question is therefore whether combining image evidence, trait-based identification, and language-model synthesis improves species-level mushroom identification compared with using each evidence source as a standalone method. This is evaluated through a benchmark suite that compares the standalone methods with unified pipeline variants under standard, confusing-lookalike, and OOD-labelled image conditions.

1.2 Exploratory market conditions and analysis

An exploratory analysis of consumer-facing mushroom identification applications was conducted to understand how current systems present identification results and to identify potential limitations in existing solutions. The purpose of this analysis was not to perform a full systematic review, but rather to provide market-oriented context for the thesis.

The analysis was carried out using the Apple App Store. Two search terms were used: *Mushroom identification* and *Svampguide*. These searches returned 169 and 33 results, respectively. Since the App Store does not provide sorting by download count, popularity, or number of active users, the selection was based on the ranking order returned by the platform's search algorithm at the time of the search. This introduces a limitation, as the exact ranking mechanism is not publicly available and may vary over time, location, and user account settings.

From the initial search results, 50 applications were selected for closer inspection. Applications that required mandatory account creation, subscription payment, or other paywall restrictions before core identification functionality could be tested were excluded. This exclusion criterion was applied to ensure that each application could be evaluated under comparable conditions. After filtering, 25 applications remained available for analysis.

Each application was tested using the same input image of a chanterelle, *Cantharellus cibarius*. Using a common image allowed the outputs of different applications to be compared under consistent input conditions. The analysis focused on observable application behaviour rather than internal implementation details, since the underlying models and technical pipelines were not disclosed by the application providers.

The following aspects were recorded for each application: the type of identification result provided, whether a confidence score was shown, whether multiple candidate species were presented, whether explanatory information was included, whether the application referred to external or reference-like species descriptions, and whether the system appeared to rely primarily on image-based input. Particular attention was given to the degree of transparency provided regarding the identification method.

The analysis showed that many applications describe their functionality using general terms such as "advanced AI" without specifying the techniques, models, training data, or validation procedures used. This lack of methodological transparency makes it difficult for users to assess the reliability, limitations, and safety of the identification results. In addition, many applications produced highly similar outputs, typically consisting of species names, generic descriptions, confidence scores, and accompanying reference images. These observations suggest that several applications may rely on image-based classification combined with static species information, although this cannot be confirmed without access to their internal implementations.

Overall, the observed applications appeared to depend primarily on visual input. Limited support was found for incorporating additional contextual information, such as user-provided morphological traits, habitat, smell, location, or descriptive uncertainty. This indicates that many current consumer-facing systems follow a largely single-modality approach, where image classification is the dominant mechanism for identification.

This limitation is relevant because mushroom identification can be difficult when visual information is incomplete, ambiguous, or affected by lighting, occlusion, camera angle, or background noise. These observations support the multimodal approach investigated in this thesis, which combines image-based recognition, trait-based identification, and language-model-based synthesis. The purpose of this design is to investigate whether combining complementary evidence sources can improve robustness and interpretability compared with relying on image classification alone.

The exploratory market analysis therefore provides practical motivation for the research question and supports the selection of the identification components evaluated in the proposed system.

1.3 Research Question

This thesis is guided by one primary research question:

1. How does combining machine learning, trait based identification and large language model classification affect mushroom identification as a whole compared to using each as a standalone method?

1.4 Contributions

The thesis makes the following contributions.

- A mushroom identification pipeline was implemented using several standalone evidence sources. These include an EfficientNet-B3 convolutional neural network for image-based species classification, a YOLOv8-assisted

trait-extraction pipeline, a structured trait-database comparator, a programmatic identification key navigation engine and an LLM-based synthesis component.

- A benchmark suite was constructed to compare standalone identification methods with unified synthesis variants. The benchmark contains 30 mushroom specimens divided into standard, confusing-lookalike, and OOD-labelled cases.
- The thesis evaluates whether combining image-based classification, trait-based identification, and LLM-based synthesis improves species-level mushroom identification compared with using each evidence source as a standalone method.

1.5 Thesis Outline

Section 2 presents the theoretical and practical background for the work. Section 3 describes the dataset, identification pipeline, implementation choices, and evaluation procedure. Section 4 reports the benchmark results. Section 5 discusses the results in relation to the research question. Section 6 concludes the thesis.

2 Background

This section provides the theoretical and practical background needed to understand the identification pipeline evaluated in this thesis. It introduces mushroom identification as a trait-based task and describes the artificial-intelligence methods used in the project, including convolutional neural networks (CNN) for image-based recognition, YOLOv8-based segmentation for isolating mushroom regions, and large language models for language-based synthesis. The section also explains how these methods relate to the broader research question of whether multiple evidence sources can improve species-level mushroom identification.

2.1 Mushroom Identification as a Knowledge-Intensive Task

Traditional mushroom identification depends on a combination of visual observation and domain knowledge. Field guides describe species in terms of observable traits such as cap shape, cap colour, gill attachment, stem characteristics, habitat, and season. They also emphasize dangerous confusions between edible and toxic species. In this project, *Nya svampboken* served as the main domain source for species descriptions, trait definitions, and lookalike relationships[9]. The field-guide perspective is relevant because it motivates why image-only prediction is sometimes insufficient: a photograph may capture colour and shape, but it may omit underside structure, smell, growth substrate, or developmental stage.

2.2 Species Knowledge and Identification Keys

Species knowledge can be represented in different structured forms, including identification keys and trait matrices. An *identification key* is a hierarchical structure in which each internal node represents a diagnostic character. For example, in mushroom identification, one node may ask whether the specimen has gills. If the answer is yes, the identification follows one branch; otherwise, it follows another branch. A *dichotomous key* is a common form of identification key in which each node has two possible branches. However, identification keys can also be polytomous[10]. Traversing an identification key progressively removes taxa that are incompatible with the observed character states until a species, or a smaller set of candidate species, is reached.

A *trait matrix* represents taxa as rows and characters as columns. Each cell stores the character state of a taxon, such as gill attachment, cap colour, spore-print colour, or habitat. An observed specimen can then be compared with the rows of the trait matrix, for example by counting matching and conflicting character states. If traits are missing or ambiguous, several taxa remain possible, making the identification becomes less certain.

Trait-based identification uses observable mushroom characters as structured input to the identification process. Examples of such characters are cap colour, gill attachment, stem morphology, and habitat. Identification keys are interpretable because each step can be traced to an observable character state. They can also provide an identification route when a photograph is missing or uninformative. In a multimodal setting, trait-based evidence can complement image-based recognition because it supplies independent morphological evidence against which image-based predictions can be compared.

2.3 Image-Based Recognition

A *convolutional neural network* (CNN) is a neural network architecture commonly used for pattern recognition in images[3]. In image classification, a CNN receives an image as input and produces a prediction over a predefined set of classes. The network consists of several neural layers that transform the input image into increasingly abstract feature representations. Early layers often respond to local visual patterns, such as edges, colours, and textures, whereas later layers combine these patterns into higher-level representations that are useful for classification.

Training a CNN from ground up usually requires a large labelled image dataset. This requirement can be difficult to satisfy in smaller applied projects, especially when the target domain contains visually similar classes and limited training material. For this reason, transfer learning is often used. In transfer learning, a model pretrained on a large image dataset is reused and adapted to a more specific task. The pretrained model provides general visual features, while fine-tuning adjusts the model parameters to the target domain[4].

EfficientNet is a family of CNN architectures designed to balance network depth, width, and input resolution in a systematic way[5]. Such architectures are relevant for image-based recognition tasks because they can provide strong classification performance while limiting computational cost.

For mushroom identification, CNN-based image recognition is useful because it can provide a rapid initial species estimate directly from a photograph. However, the robustness of image-based classification depends on how well the training data represents the variation present in real field images. Factors such as background clutter, lighting

variation, partial occlusion, image quality, viewing angle, and visual similarity between confusable species can reduce performance when they differ from the distribution represented in the training data. A CNN may also learn correlations in the training images that are not part of the mushroom itself, such as background objects or recurring photographic conditions, if the dataset does not contain enough variation.

2.4 Artificial Intelligence and AI systems

Artificial intelligence (AI) is a broad field of computer science concerned with machine-based methods that infer and produce outputs from input data or encoded knowledge. According to the OECD, an AI system is a machine-based system that, for explicit or implicit objectives, infers from the input it receives how to generate outputs such as predictions, content, recommendations, or decisions [11]. The OECD also emphasises that there is no single clear technical boundary between AI and non-AI software. Instead, AI systems can be understood through a continuum of features commonly associated with artificial intelligence, such as inference, autonomy, and adaptiveness [11].

This broad definition is useful in this thesis because the implemented components differ in architecture, input type, and purpose. The image classifier, such as the CNN used in this project is a deep-learning computer-vision model that predicts mushroom species from photographs. The segmentation model is also based on deep learning, however instead of classification it instead predicts which image pixels belong to the mushroom region. The multimodal large language model (MLLM) produces a interpretation from the evidence supplied to it. These components are treated as AI methods because they infer outputs from input data rather than only executing manually specified instructions.

AI includes several technical approaches. Machine learning is a subset of AI in which models aim to improve their performance through exposure to data rather than through explicit instructions for every possible case [11]. This includes unsupervised methods, such as k-means clustering, which identify patterns in unlabelled data, and supervised methods, such as convolutional neural networks (CNNs), which learn from labelled examples. Deep learning is a subset of machine learning based on neural networks with many layers. In this project, the EfficientNet-B3 classifier and the YOLOv8-based segmentation model are examples of deep-learning methods used for image analysis, while k-means is used as a supporting unsupervised method for colour clustering during trait extraction.

Not every algorithm in the project is treated as an AI method. Classical computer-vision operations, such as colour-space conversion, thresholding, edge detection, and Hough line detection, are deterministic image-processing methods. They are useful for extracting visible traits, but their behaviour is mainly defined by manually specified operations rather than by learned model parameters. The trait-based part of the project combines a learned segmentation model with deterministic image-processing algorithms and structured species knowledge.

2.5 Large Language Models and Multimodal AI

A *large language model* (LLM) is a deep-learning model, commonly based on the transformer architecture, that processes language as sequences of tokens and models contextual relationships between them [6;12]. Modern LLMs can generate text, classify text, extract structured information from free-form descriptions, and follow task-specific response formats [7]. They make LLMs relevant in applications where information is expressed in natural language or where several pieces of evidence can be represented textually and interpreted together.

A *multimodal large language model* (MLLM) extends a language model by processing and integrating more than one modality, such as text, images, audio, or video [13]. In a vision-language setting, this means that the model can condition its generated language output not only on textual prompts, but also on visual input. This capability is relevant in this thesis because the MLLM is used as the main synthesis component in the unified pipeline. It receives mushroom images together with evidence produced by other modules, such as image-based predictions, extracted traits, and structured trait information, and generates a language-based classification.

The relevance of the MLLM therefore lies in its ability to combine visual and textual evidence within a single model interaction. This differs from a text-only LLM, which can only synthesise evidence after image information has first been converted into text by other components. In this thesis, the MLLM is evaluated both as a standalone identification method and as a synthesis mechanism for investigating whether combining image evidence, trait-based information, and structured species knowledge improves mushroom identification compared by using each evidence source independently.

However, MLLM outputs can be sensitive to prompt design and to the quality and relevance of the supplied input [14]. Furthermore, although an MLLM can generate an explanation for its prediction, this explanation should not be assumed to provide a faithful account of the factors that caused the model to produce that prediction. Instead, it should be understood as generated language conditioned on the prompt, image, and supplied evidence [15]. This

limitation is relevant to mushroom identification because a plausible explanation does not guarantee that the species prediction is correct or that the model relied on the same diagnostic traits that a human expert would use [15;16].

2.6 Image Segmentation

Image segmentation is the task of separating an object of interest from the rest of an image. A common distinction in segmentation is between *things* and *stuff*. Things are countable objects in an image, such as cars, people, or mushrooms. Stuff refers to less clearly bounded background regions, such as grass, soil, sky, or road surfaces. This distinction is useful because some image regions are naturally treated as separate objects, whereas other regions are better treated as continuous background areas.

Image segmentation is related to image classification and object detection, but these tasks provide different levels of spatial information. Image classification assigns a class label to an entire image. Object detection localises an object of interest by predicting a bounding box around it. A bounding box gives the approximate area of the object, but it does not describe the exact object shape. Segmentation provides a more detailed representation by assigning pixels to the object or region of interest.

Image segmentation is commonly divided into semantic segmentation and instance segmentation. Semantic segmentation assigns a class label to each pixel, but does not distinguish between separate objects of the same class. For example, if several mushrooms of the same class appear close together, semantic segmentation may represent them as one mushroom region. Instance segmentation separates individual object instances and is therefore useful when each object should receive its own mask.

Instance-segmentation models can be divided into single-stage and two-stage approaches. Single-stage models, such as models in the You Only Look Once (YOLO) family, perform object localisation, classification, and mask prediction in one forward pass. Two-stage models, such as region-based convolutional neural networks, first generate candidate object regions and then refine the classification and segmentation within those regions. The usual trade-off is that single-stage models are often faster, whereas two-stage models may provide more refined predictions [17].

In mushroom identification, segmentation is useful because field photographs often contain background material such as soil, leaves, grass, hands, baskets, or shadows. If later colour, shape, and texture analysis is performed on the full image, these background regions may distort the extracted traits. Instance segmentation is therefore relevant because it can restrict downstream analysis to the mushroom region rather than the entire photograph. The YOLOv8 implementation used in this project is described in Section 3.4.

2.7 Classical image processing methods

Image analysis of mushroom photographs can be performed using both machine-learning methods and classical image-processing methods. In this project, classical methods are used to extract visible traits such as colour, shape, edges, contours, and surface texture. These methods do not learn directly from training data in the same way as a neural network. Instead, they apply predefined transformations and rules to the pixel values in an image.

The main classical image-processing concepts used in this project are:

- colour spaces and HSV representation,
- clustering and colour quantisation,
- mask-based region analysis,
- contour-based shape analysis,
- edge, line, and blob detection,
- threshold-based heuristic classification.

A colour space defines how colour information is represented numerically. A common representation is RGB, where each pixel is described by red, green, and blue components, usually with values in the range 0–255. RGB is suitable for storing and displaying images, but it can be less convenient for colour analysis because colour and brightness are combined in the same three channels. For example, the RGB values of a mushroom photographed in bright sunlight may differ substantially from the RGB values of the same mushroom photographed in shadow.

HSV is an alternative colour representation that separates colour information from brightness more explicitly. In HSV, hue describes the general colour type, such as red, yellow, green, or blue. Saturation describes how strong or pure the colour is, while value describes the brightness of the colour [18;19]. This separation makes HSV useful for approximate colour-based trait extraction, because colour labels can be based mainly on hue and saturation while brightness can

be considered separately. However, HSV does not remove all effects of lighting, and strong shadows, highlights, or camera white balance can still influence the result.

2.7.1 Clustering and colour quantisation

After converting image pixels from RGB to HSV, the remaining question is how colour analysis can be performed for a selected pixel area. One approach is to use K-Means clustering. K-Means groups similar pixels into a predefined number of clusters based on their colour values. The algorithm begins by selecting initial cluster centres and then assigns each pixel to the nearest centre according to a distance measure. The cluster centres are then updated based on the pixels assigned to them, and this process is repeated until the clusters stabilise [20].

In colour analysis, each cluster centre can be interpreted as a representative colour for part of the selected image region. The cluster containing the largest number of pixels is treated as the dominant colour, while smaller clusters may represent secondary colours, shadows, highlights, or noise [21]. The number of clusters must therefore be chosen carefully. Too few clusters may merge visually distinct colours, while too many clusters may overrepresent minor variations caused by lighting, texture, or image noise.

2.7.2 Mask-based region analysis

In image analysis, a binary mask can be used to restrict measurements to a specific region of interest. A binary mask is an image where pixels belonging to the object or part of interest are marked as foreground, while all other pixels are treated as background. In this project, segmentation masks are used to limit classical image-processing measurements to mushroom parts such as the cap, stem, or underside. This reduces the influence of surrounding background pixels when performing image analysis such as estimating colour, shape, brightness, and texture.

2.7.3 Contour-based shape analysis

Shape can be analysed by extracting the contour of a segmented object. A contour describes the boundary of a foreground region in a binary mask. From this boundary, geometric descriptors such as area, perimeter, bounding-box dimensions, aspect ratio, circularity, convexity, and solidity can be computed [22]. These descriptors provide numerical summaries of the object's form.

For example, circularity measures how close a contour is to a circle, while aspect ratio compares the width and height of the object. Solidity compares the contour area with the area of its convex hull and can indicate whether the shape contains indentations or irregular boundaries. Such descriptors can be used to map continuous shape measurements to discrete labels, such as round, elongated, flat, convex, or irregular.

2.7.4 Edge, line, and blob detection

Edges are image locations where pixel intensity changes sharply. Edge detection is therefore useful for identifying boundaries, ridges, texture, and other local structures. The Canny edge detector is a common method for detecting such edges [23]. In texture analysis, the density of detected edges within a segmented region can be used as a rough indicator of surface complexity. A smooth surface will usually produce fewer edges, while a fibrous, scaly, or irregular surface may produce more edges.

Line detection can be used when the relevant structure is approximately linear. The Hough transform is a classical method for detecting straight lines in an edge image [24]. In mushroom analysis, this can provide evidence for structures such as gills or ridges on the underside of the cap.

Blob detection is used to find compact regions that differ from their surroundings. Small circular blobs can be useful cues when analysing pore-like structures [25]. In this project context, edge, line, and blob-based measurements provide simple visual cues that support trait extraction, but they remain sensitive to image quality, lighting, occlusion, and segmentation errors.

2.7.5 Threshold-based heuristic classification

Classical image-processing measurements often produce continuous numerical values, such as edge density, circularity, brightness, or aspect ratio. To use these values as categorical traits, they can be mapped to labels using manually defined thresholds. For example, a high circularity value may indicate a round object, while a low circularity value may indicate an irregular object. Similarly, a high edge density may indicate a rougher texture.

This type of threshold-based classification is simpler than a learned classifier and does not require labelled training data for each trait. However, it is also less flexible and depends on manually chosen rules. As a result, the extracted traits should be interpreted as approximate visual cues rather than definitive biological measurements.

2.8 Combining Multiple Evidence Sources

Species identification can depend on several forms of evidence. In mushroom identification, visual appearance may be combined with morphological traits, habitat, season, smell, texture, and other contextual information. A single photograph may therefore be insufficient, especially if important characters such as the underside of the cap, stem base, or surrounding habitat are missing or unclear.

Computational identification systems can also combine multiple evidence sources. Image classifiers can learn visual patterns from photographs, while trait-based methods can represent explicit morphological characters such as cap colour, gill attachment, stem shape, and habitat. These sources have different strengths and limitations. Image-based models may be affected by lighting, background clutter, viewing angle, and dataset bias, whereas trait-based methods depend on the availability and correctness of observed or extracted traits.

Combining evidence sources can make an identification process more robust by allowing one source of evidence to support, contradict, or compensate for another. For example, an image classifier may suggest a visually similar species, while a trait-based key can reject candidates that conflict with observed morphological characters. Similarly, structured trait comparisons can highlight dangerous lookalikes even when the top image-based prediction appears plausible.

Large language models and multimodal language models provide one possible way to synthesise heterogeneous evidence. They can receive textual descriptions, extracted traits, and in some cases images, and produce a natural-language interpretation of the combined input. However, adding more information to the input does not guarantee a better identification. Prior work on long-context language models shows that models may not use all parts of a long prompt equally well, and that relevant information can be harder to use when it appears in less salient parts of the context [26]. Models can also be distracted by irrelevant or misleading context [27]. This means that evidence-rich prompts may improve performance when the information is relevant and well structured, but may reduce reliability when the prompt becomes long, noisy, contradictory, or difficult to prioritise.

3 Method

This section describes the benchmarked mushroom-identification pipeline and the comparative methodology used to evaluate the research question, namely whether combining image classification, trait-based reasoning, and multimodal language-model synthesis improves species identification compared with the standalone methods. Section 3.1 introduces the Benchmark 5 design and the A1, A2, B1, and B2 evaluation variants. Section 3.2 describes the dataset construction and image sources. The following component sections then introduce the EfficientNet-B3 image classifier in Section 3.3, YOLOv8 part segmentation in Section 3.4, classical visual trait extraction in Section 3.5, and the trait-based identification-key and database components in Section 3.6. Section 3.7 describes the standalone multimodal LLM baselines, while Section 3.8 describes the unified multimodal LLM synthesis stage. Section 3.9 describes prompt design. Finally, Sections 3.9.1–3.10.1 describe input construction, fallback behaviour, response parsing, agreement evaluation, and the benchmark evaluation procedure.

3.1 Benchmark Design Overview

The project was initially conceived as a mushroom-identification mobile application; however, the contribution evaluated in this thesis is the comparative analysis of identification accuracy between *standalone* knowledge sources and a *unified multimodal LLM* that synthesises those sources. All interface components (Flutter frontend, Java Spring Boot proxy, camera integration, local history, and settings) are therefore outside the evaluation scope. The benchmark invokes the Python identification components directly and records their outputs so that standalone methods and the combined prediction can be compared under controlled conditions.

The central research question is operationalised through two benchmark settings: standalone evidence sources and unified evidence synthesis. Within these settings, trait-based methods are evaluated under two input conditions. The first condition uses traits automatically extracted from the specimen photographs. The second uses oracle traits retrieved from `species_traits.xml` using the ground-truth species label. These oracle traits represent idealised species-level reference information rather than characteristics measured directly from the photographed specimen.

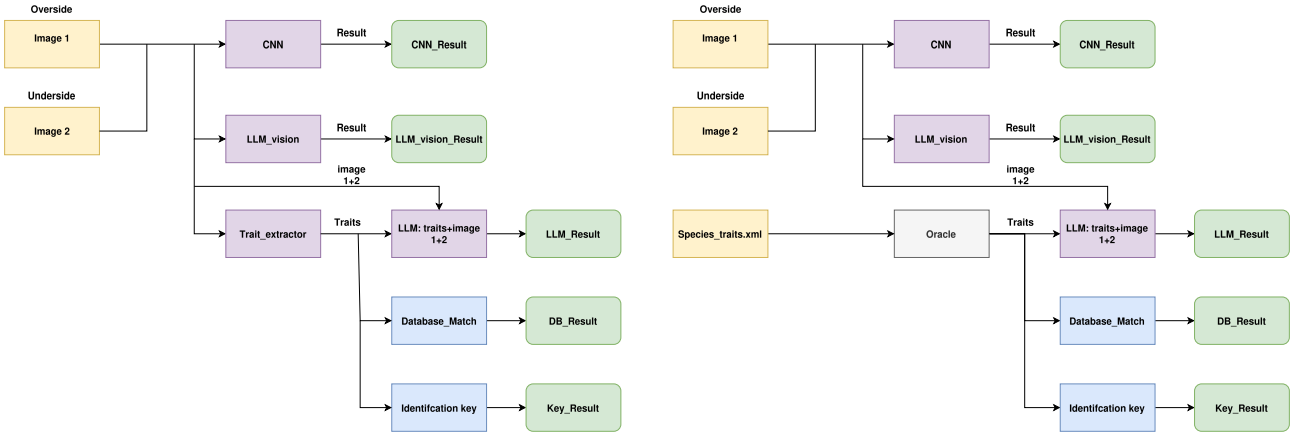
System A1: Standalone methods with extracted traits. System A1 evaluates the individual identification components separately under realistic input conditions. These include the EfficientNet-B3 CNN classifier, the image-only Multimodal LLM (MLLM), the MLLM supplied with automatically extracted traits, the identification-key traversal, and the trait-database comparator.

System A2: Standalone methods with oracle traits. System A2 evaluates the same standalone trait-dependent methods, but replaces automatically extracted traits with species-level oracle reference traits retrieved from `species_traits.xml` using the ground-truth species label. This setting evaluates how the downstream methods behave when supplied with more complete reference trait information rather than automatically extracted image traits.

System B1 (Unified multimodal LLM synthesis). System B1 evaluates the unified pipeline using automatically extracted traits. The MLLM receives the original images, the CNN prediction, the extracted trait dictionary, the validated identification-key result, and a trait-database validation report. In this setting, the database comparator does not rank all species independently. Instead, it validates the species proposed by the key traversal, or the CNN top prediction if no key conclusion is available.

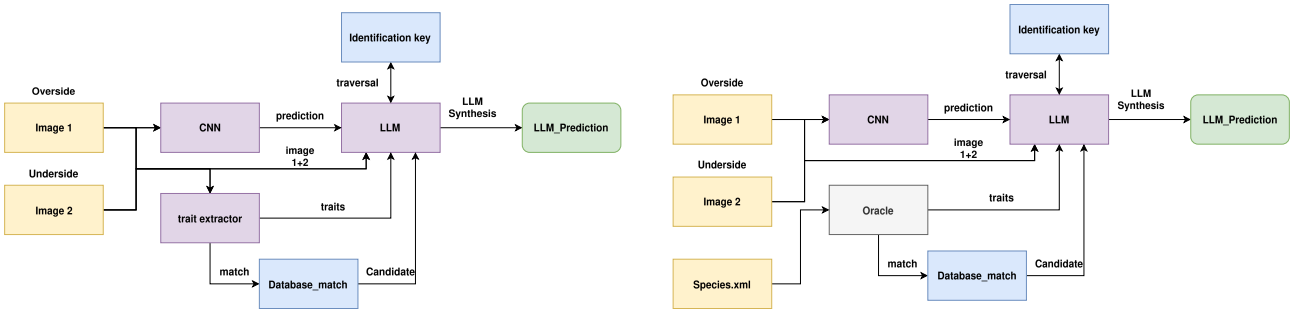
System B2 (Unified multimodal LLM synthesis with oracle traits). System B2 evaluates the same unified pipeline as B1, but replaces the extracted traits with species-level oracle reference traits from the reference trait data. This tests whether supplying more complete reference trait information improves the unified synthesis result.

Figures 1 and 2 use a shared colour scheme to distinguish the roles of the pipeline components. Yellow boxes represent input data, purple boxes represent machine-learning components, blue boxes represent rule-based reasoning, and green boxes represent the final result.



(a) A1: extracted traits **(b)** A2: oracle traits

Figure 1: System A standalone benchmark pipelines. (a) A1 uses computer-vision trait extraction. (b) A2 replaces the trait extractor with ground-truth oracle traits.



(a) B1: extracted traits **(b)** B2: oracle traits

Figure 2: System B unified benchmark pipelines. (a) B1 uses computer-vision trait extraction. (b) B2 replaces the trait extractor with ground-truth oracle traits.

The benchmarked pipeline for System B consists of four sequential stages that feed into the LLM synthesis step:

1. **Visual trait extraction and CNN prediction:** The input images are analysed by a part-aware classical computer-vision trait extractor and by a pretrained image classifier EfficientNet-B3. The extractor uses a fine-tuned YOLOv8-seg model to segment the mushroom into anatomical parts (cap, stem, underside), then extracts a structured `visible_traits` descriptor containing both part-specific traits (cap colour, stem ring, hymenophore type).
2. **Identification-key traversal:** A Swedish-language polytomous identification key, stored in `key.xml`, is traversed using available trait evidence. Image-derived traits are used to answer key questions when the cue is sufficiently specific.
3. **Trait-database validation:** If the key traversal reaches a species conclusion, that candidate is validated against `species_traits.xml`. If the key traversal is inconclusive, the CNN top prediction is validated instead. Lookalike data from `lookalikes.csv` are retrieved when available.
4. **Multimodal LLM synthesis:** A vision-capable MLLM receives the original above- and below-view photographs together with the structured outputs from stages 1–3. The MLLM performs its own direct visual analysis and then critically evaluates the subsidiary evidence to produce a final species prediction, confidence estimate, and explanatory reasoning.

The following subsections describe the dataset, the knowledge-source components, the standalone vision-language baselines, the multimodal LLM synthesis mechanism, and the evaluation procedure.

3.2 Dataset Construction

The benchmarked components use three main data and knowledge sources:

- **CNN training images:** The training data for the EfficientNet-B3 convolutional neural network (CNN) were collected from iNaturalist. Two filters were applied during data collection: only mushroom taxa were selected, and observations were restricted to Sweden, with Scandinavia as a fallback when Swedish samples were scarce. The resulting curated image subset contains 640 images distributed across the 16 species handled by the neural classifier, that is, 40 images per species.
- **YOLOv8 training images:** Images for training the YOLOv8 segmentation model were collected from the Global Biodiversity Information Facility (GBIF), Danske Svampeatlas, and iNaturalist. The same filters used for the CNN training data were applied to the YOLOv8 image dataset.
- **Identification key:** The structure of `key.xml`, used as the identification key, is based on *Vår Svampbok* [28].

3.3 CNN-Based Image Classification

The image-recognition component uses a convolutional neural network (CNN) based on an EfficientNet-B3 backbone[5]. The model is implemented with the PyTorch Image Models (`timm`) library and initialised with ImageNet-pretrained weights. It is then fine-tuned on a curated dataset containing sixteen mushroom classes: *Birch Polypore*, *Black Trumpet*, *Bay Bolete*, *Cauliflower Mushroom*, *Chanterelle*, *Clustered Coral*, *Destroying Angel*, *False Chanterelle*, *Fly Agaric*, *King Agaricus*, *Lakrits Milkcap*, *Mandel Milkcap*, *Pale Coral Fungus*, *Porcini*, *Puffball*, and *Shaggy Inkcap*.

Input images are resized to 300×300 pixels and normalised before being passed to the network. The CNN outputs class scores for the sixteen supported mushroom classes. These scores are converted into a softmax distribution, which is the internal prediction of the CNN that gives a confidence value for each class. The class with the highest confidence is stored together with its confidence value in an `ml_prediction` structure.

The EfficientNet-B3 model was trained using a two-phase transfer-learning procedure. In the first phase, the pretrained EfficientNet-B3 backbone remains frozen while only the classification head is trained. In the second phase, the full network is fine-tuned end-to-end using a reduced learning rate. During training, the model records training loss, training accuracy, validation loss, and validation accuracy; validation accuracy is used as the main model-selection criterion. After each epoch, the validation accuracy is compared with the best validation accuracy observed so far. If the validation accuracy improves, the current model weights are saved. If the validation accuracy does not improve for five consecutive epochs, early stopping terminates training. The saved model weights, stored as `cnn_weights.pt`, are used in the pipeline.

In System B, the CNN prediction is not treated as authoritative. Instead, it is passed to the multimodal LLM as one subsidiary signal among several, tagged with a warning that CNN classifiers can be overconfident on unusual lighting,

odd angles, or species outside its sixteen-class training set.

In the standalone CNN benchmark, the CNN was applied to both the above-view and below-view images when both were available. The benchmark then kept the prediction with the higher top confidence. The two softmax distributions were not averaged or otherwise merged. This differs from the CNN signal used inside the unified B-family pipeline, where one image is selected before CNN inference and the CNN is run once on that selected image.

3.4 Image Segmentation with YOLOv8

Before trait extraction can be applied, the pixels of interest must be separated from the background. Instance segmentation is used for this purpose. The system employs YOLOv8n-seg, the nano variant of the You Only Look Once version 8 (YOLOv8) segmentation model, fine-tuned on annotated mushroom photographs. For each detected object instance, the model outputs an anatomical class label, a bounding box, a confidence score, and a segmentation mask indicating which pixels belong to the detected part.

Annotations were created using the Roboflow platform[29]. The Smart Polygon tool, which is backed by Segment Anything Model 2 (SAM2)[30], was used to generate initial per-object masks of mushroom fruiting bodies. These masks were then manually reviewed, corrected where necessary, and assigned one of three anatomical class labels: *cap* for the upper surface of the pileus, *stem* for the stipe supporting the fruiting body, and *underside* for the hymenium-bearing surface. The annotated dataset contains 309 images with masks across the three anatomical classes, plus 55 negative images without annotated mushroom parts, for a total of 364 images. The negative images were included to expose the model to background appearances such as leaves, soil, trees, hands, and other non-mushroom regions.

The labelled masks provide the supervised training signal for YOLOv8 by defining which image regions correspond to each anatomical part class. During model development, a dataset is normally divided into training, validation, and test subsets. The training subset is used to update the model weights, The validation subset is used during training to monitor whether the model generalises to images it has not directly learned from. This can reveal overfitting, for example if the model begins to rely on background cues rather than anatomical mushroom structures. In this project, Roboflow exported the annotated images in a single split, so the training, validation, and test partitions were instead assigned randomly in Google Colab before YOLOv8 training.

YOLOv8 training was performed in Google Colab, a cloud-based Jupyter notebook environment that provides access to Python, GPU acceleration, and common machine-learning libraries[31]. The annotated Roboflow export was uploaded to the Colab runtime, extracted, and used with the Ultralytics YOLO training API. After training, the resulting checkpoint `best.pt` was downloaded and copied into the local project for inference.

3.5 Classical Visual Trait Extraction

The classical image-processing operations were implemented using OpenCV. Colour conversion used `cv2.cvtColor`, contour extraction used `cv2.findContours`, edge detection used `cv2.Canny`, line detection used `cv2.HoughLinesP`, and blob detection used `cv2.SimpleBlobDetector`[32].

When YOLOv8 segmentation has identified the relevant anatomical regions, the trait extractor applies classical computer-vision methods only to the masked area of interest. This reduces interference from background pixels such as soil, leaves, grass, hands, and shadows. Each accepted YOLOv8 part mask is handled by a dedicated analyser, so that the extracted traits are attributed to the correct mushroom part rather than to the full photograph.

Cap Analyser extracts dominant and secondary cap colour using K-Means clustering ($k = 4$) on HSV pixels inside the cap mask. It estimates cap shape from contour measurements on the binary cap mask, including aspect ratio, circularity, central-depression score, and contour complexity. Cap surface texture is estimated from Canny edge density within the cap mask, with high edge density interpreted as coarse or scaly texture.

Stem Analyser extracts stem colour, brightness, thickness, aspect ratio, surface texture and ring presence. Colour is computed with the same masked HSV K-Means procedure as the cap analyser. Brightness is measured from the average HSV value channel inside the stem mask. Thickness and aspect ratio are derived from the stem mask bounding box. Ring presence is estimated by measuring grayscale intensity variation in the upper 30 % of the stem bounding box.

Underside Analyser extracts underside colour and classifies the hymenophore type as *gills*, *pores*, *ridges*, *teeth*, or *unknown*. The classification uses a composite heuristic: Hough-line count is used for gills and ridges, circular blob detection is used for pores, and Canny edge density is used for tooth-like structures. A confidence value is derived from the strongest heuristic score and reduced when competing scores are close.

Whole-Mushroom Analyser supports puffball-like specimens and other cases where the specimen is better treated as one compact fruiting body rather than as separate cap, stem, and underside parts. It analyses the whole-body union mask and extracts whole-body colour, brightness, surface texture, roundness, and aspect ratio. Surface texture is estimated from edge density and mapped to smooth, warty, or spiny descriptions.

3.5.1 Colour representation and clustering

All colour analysers operate in HSV space rather than RGB. HSV separates chromatic information, represented by hue and saturation, from overall brightness, represented by value. This makes the colour estimate less dependent on raw RGB intensity than direct RGB comparison, although shadows, highlights, and camera white balance can still affect the result. Because many mushrooms exhibit two-tone or mottled colouration, a single dominant colour is often insufficient. K-Means with four clusters captures the main colour mass plus up to three secondary patches. Four was chosen because often mushrooms only have either one dominant and secondary colour whilst remaining clusters catch shadows, highlights or mottling. Neutral colours (white, grey, black) are handled through saturation and brightness thresholds rather than hue ranges, because their hues are undefined or unreliable. For colour extraction, if a part mask is too small to provide enough pixels, the masked colour analyser falls back to the legacy whole-image colour procedure. This fallback downsamples the full image to 128×128 pixels and applies K-means with five clusters to estimate dominant and secondary colours.

3.5.2 Shape heuristics and threshold calibration

Cap shape is discretised into labels such as convex, flat, bell-shaped, depressed, funnel-shaped, wavy, and irregular by thresholding geometric descriptors computed from the binary cap-mask contour. The thresholds were chosen by manual inspection of contour statistics on a small calibration set: high circularity with near-unity aspect ratio maps to convex, a wide contour with low circularity maps to flat, a strong central-depression score supports depressed, low aspect ratio with moderate circularity maps to bell-shaped, and high contour complexity with low circularity maps to irregular.

3.5.3 Hymenophore classification with competing heuristics

The underside classifier must distinguish gills, pores, ridges, and teeth from a single photograph taken at variable angles and with variable focus. No single feature is reliable in isolation. Hough-line density captures the parallel elongated structures typical of gills and ridges, circular blob detection captures the small rounded openings of pores, and high Canny edge density without dominant line structure suggests tooth-like structures. The analyser scores all four hypotheses and selects the highest if it exceeds a minimum margin over the runner-up. When two scores are close, confidence is reduced and the type may be reported as *unknown*. This conservative design reflects that underside photographs are often partially occluded, blurred, or unevenly lit, and an incorrect hymenophore label would mislead the downstream key traversal.

3.5.4 Puffball case and the whole-body path

The whole-body mask is produced as a pixel-wise union of all accepted part masks, so whole-body traits are available for every segmented specimen. Puffballs and other compact fruiting bodies lack the classical cap–stem–underside anatomy; forcing them through the part-aware analysers alone would produce meaningless values (e.g., a missing stem mask or a spurious underside label). The extractor therefore labels the morphology case *puffball* only when the segmentation output contains exactly one cap part (no separate stem or underside) *and* the whole-body union mask is compact and round (circularity above 0.75 and aspect ratio below 1.3). In all other situations the case-driven routing treats the specimen as a classical mushroom. Roundness is measured by contour circularity and aspect ratio distinguishes spherical from pear-shaped bodies. This design avoids inventing traits that do not exist and provides the downstream components with descriptive evidence appropriate to the actual morphology.

3.5.5 Morphology case derivation

After the part analysers have run, the extractor derives a morphology case from the detected parts and shape evidence. A cap-only, round, compact body is treated as *puffball*. A cap together with a stem or underside is treated as a classical mushroom; within this group the cap-shape label is used to subdivide the case into *classical_convex* (convex, flat, or bell-shaped caps) and *classical_concave* (funnel-shaped or depressed caps). If none of these conditions match, the case falls back to *classical_unknown* or *uncertain*.

3.5.6 Output schema: part-aware and legacy fields

The final `visible_traits` output contains both part-aware fields (`cap_color`, `stem_color`, `hymenophore_type`, `stem_ring`, `stem_surface`, `puffball_surface`, `morphology_case`) and legacy compatibility fields (`dominant_color`, `secondary_color`, `cap_shape`, `surface_texture`, `has_ridges`, `brightness`). The legacy fields are not produced by a separate whole-image analyser; they are derived from the part-aware outputs. For example, `dominant_color` is taken from `cap_color` with `whole_color` as a fallback, while `has_ridges` is set when `hymenophore_type` equals `ridges`. This dual-field design preserves compatibility with downstream components originally written for an earlier whole-mushroom segmentation pipeline while still exposing the richer part-aware data for newer modules.

3.5.7 Confidence scores as quality gates

The extractor stores a heuristic confidence score alongside each part-aware trait. These scores are *not* calibrated probabilities; a value of 0.80 should not be interpreted as an 80 % chance of correctness. Instead, each score summarises reliability indicators from the segmentation and analysis stages: mask size (area ratio), coherence (fragmentation count), hole count (hole ratio), border contact, YOLOv8 detection confidence, and analyser-specific signal strength. Cap traits are weighted more strongly than stem traits (detector score 0.85 versus 0.70), while underside traits inherit the hymenophore classifier's internal confidence. Downstream components use these scores as quality gates: the key traversal only auto-answers questions when the relevant trait confidence is high, and the trait-database comparator only treats conflicting part-aware traits as hard conflicts when their confidence exceeds a configurable threshold. The scores therefore function as evidence-quality metadata rather than species-identification probabilities.

3.6 Trait-Based Identification Key and Database

Identification key This project uses an identification-key representation as the main structured trait-based identification mechanism. This choice was made because the selected characters are treated as sequential decision points, where each observation narrows the remaining set of possible taxa. During traversal, taxa that do not match the current observation are removed from consideration. For example, if the observed mushroom has attached gills, taxa described as having free gills are excluded from the remaining candidates.

A trait-matrix representation would instead allow species to be compared by the overall number of matching and conflicting traits. This can be useful when the available observations are incomplete or ambiguous, but it does not impose the same stepwise decision structure as an identification key. Since the project is based on field-guide-style trait reasoning, the identification-key representation was used for the main traversal procedure, while structured trait data were used as supporting information during comparison and warning generation.

Trait-based reasoning follows two complementary mechanisms: expert identification-key traversal and trait-database comparison.

Identification-Key Traversal. The key is stored as an XML-encoded polytomous tree derived from the Swedish field guide *Vår Svampbok* [28]. Each internal node poses a diagnostic question, such as *"Hur ser svampen ut?"*, and each branch corresponds to a possible answer. Traversing the tree excludes taxa that are incompatible with the observed character states.

Precheck conclusions. In addition to ordinary XML traversal, the key engine contains a precheck mechanism for a small number of CNN-suggested species that are not represented as normal traversable leaves in `key.xml`. Before traversal begins, the engine checks whether the CNN `ml_hint` refers to one of these species. If the CNN confidence is high enough, the engine can return that species as a precheck conclusion instead of continuing through the XML key. A precheck conclusion is therefore not the result of reaching a terminal XML leaf. It is a compatibility mechanism for species that can occur in the benchmark but are not fully covered by the XML traversal structure.

The system implements *two* traversal mechanisms, used in different benchmark configurations.

Programmatic engine (System A1 and A2). In the standalone benchmark, a deterministic engine, `KeyTreeEngine`, walks the tree question by question. It attempts to answer each node automatically from the available `visible_traits`. In A1 these traits come from the computer-vision trait extractor, while in A2 they are replaced by oracle traits from the reference trait database. For example, if the traits report `hymenophore_type = pores` and the colour ratios indicate a brown-dominant, low-red profile, the engine selects the branch for bolete-like mushrooms. If the engine reaches a question that cannot be answered from the available traits, the benchmark records the method as not covered for that specimen rather than supplying manual answers. In an interactive application, such unresolved key questions could instead be passed to the user. However, this thesis evaluates automated benchmark performance, so user-assisted traversal is excluded in order to keep the systems comparable.

The engine also accepts an `ml_hint` from the CNN. For a small number of hardcoded species–question pairs, the hint is used to auto-answer specific key questions (e.g., confirming a poroid rather than gilled hymenophore for Porcini or colour for Chanterelle), but the tree is otherwise traversed normally. Before traversal begins, a compatibility check classifies the hinted species into three tiers: (i) *exact-match* species (Chanterelle, Black Trumpet, Porcini, Other Boletus), which are traversed normally; (ii) *lookalike-only* species (Amanita virosa, False Chanterelle), which appear in the key only as toxic lookalikes under other decisions; and (iii) *unsupported* species (Fly Agaric), which are absent from the key entirely. For tiers (ii) and (iii), if the CNN confidence is ≥ 0.85, the engine bypasses traversal entirely and returns a pre-check conclusion with edibility information derived from the lookup table.

LLM navigator (System B1 and B2). In the unified pipeline, the multimodal LLM performs tree navigation. The LLM receives a compact text representation derived from `key.xml`, together with the extracted or oracle traits, CNN hint, and case classification, and returns a proposed root-to-leaf path. A `TreePathValidator` checks the returned path against the XML schema; if the path is valid and reaches a decision node, it is accepted as the traversal conclusion. If the path is invalid or incomplete, the pipeline records the deepest valid node reached and continues to database comparison and synthesis. The programmatic `KeyTreeEngine` is retained only as a fallback when the LLM backend is unavailable.

The key tree does not cover all species in the training database (e.g., *Amanita muscaria* is absent from the key), so exact traversal is impossible for species outside the tree regardless of which navigator is used.

Trait-Database Comparison. The trait-database comparator is used differently in the standalone and unified benchmark settings. In System A1 and System A2, it acts as an independent trait-based predictor: the available traits are compared against every species profile in `species_traits.xml`, and the highest-scoring species is returned as the database prediction.

In System B1 and System B2, the comparator instead acts as a candidate-validation step. If the identification-key traversal reaches a valid species conclusion, the visible traits are compared against the corresponding species profile. If no conclusive key result is available, the comparator validates the CNN top prediction instead. If neither a key conclusion nor a CNN candidate is available, the database-comparison step is skipped. In both cases, the comparator reports matched traits, conflicting traits, non-comparable traits, a trait-match score, and documented lookalikes from `lookalikes.csv`.

In System B, both the key-traversal result and the trait-database validation report are serialised and passed to the MLLM as structured text evidence. The MLLM receives the key path taken, the conclusion reached, the trait-match score, and any lookalike warnings.

3.7 Benchmark A1 and A2 MLLM

In addition to the CNN and the key–database subsystem, two standalone vision-language baselines are evaluated as part of System A. Both use the same underlying model Gemma 3, served locally via Ollama but differ in whether auxiliary trait information is provided.

3.7.1 A1: LLM capability

In the image-only condition, the model is instructed to identify the species from the above- and below-view images using a minimal mycological prompt. In the extracted-trait condition, the same image pair is accompanied by the automatically extracted `visible_traits`. The images are resized to 896 pixels on the longest side (the native resolution of the Gemma 3 vision encoder) and encoded as JPEG base64 strings before being sent to the local Ollama endpoint. The model returns a JSON-structured response containing the predicted species, confidence score, and brief reasoning.

This baseline serves two purposes. First, it provides a direct comparison against the unified pipeline, demonstrating the value (or lack thereof) of auxiliary knowledge sources. Second, it isolates the contribution of the LLM's own visual reasoning capability independent of any structured evidence.

3.7.2 A2: LLM with Oracle Traits

The A2 baseline uses the same MLLM setup as A1 but receives oracle visible-trait information alongside the images. These oracle traits are derived from `species_traits.xml` using the ground-truth species label and are appended to the prompt as a structured text block of known visible traits, including cap, gill, stem, and flesh characteristics. The model is instructed to use these traits as reference evidence when forming its prediction.

The A2 baseline reduces the visual-perception burden on the standalone MLLM and helps separate errors caused by missing or misread visual traits from errors caused by model reasoning. The gap between A1 and A2 indicates how much the standalone MLLM benefits from explicit trait information. The comparison between A2 and B2 should be interpreted more broadly: it measures how the unified evidence-integration pipeline performs when supplied with oracle traits, including the effects of key navigation, database comparison, prompt size, and final synthesis.

3.8 Unified LLM Synthesis

The unified identification pipeline, System B, uses a multimodal large language model as its central synthesis mechanism. Gemma 3 12B was selected because it could be run locally through Ollama within the available hardware and cost constraints, while still supporting multimodal image-and-text input. During synthesis, the model receives the above- and below-view mushroom images together with structured evidence from the other components. This evidence includes the CNN prediction and confidence score, the extracted or oracle traits, the identification-key traversal result, and the trait-database validation report.

The MLLM is instructed to first form its own visual interpretation from the images, then critically evaluate the subsidiary evidence, and finally produce a species prediction with confidence, reasoning, and safety warnings. This design treats the MLLM as the principal synthesis component, while the standalone methods are treated as fallible evidence sources whose outputs must be weighed against one another.

Two variants of the unified pipeline are evaluated:

- **B1 (extracted traits):** The LLM receives the `visible_traits` produced by the classical computer-vision extractor. This measures end-to-end pipeline performance under realistic field conditions where traits must be inferred from the images.
- **B2 (oracle traits):** The LLM receives ground-truth traits as structured text evidence instead of automatically extracted traits. This isolates the quality of the synthesis mechanism from the quality of the trait extractor.

3.9 Prompt Construction and Generation Settings

The final synthesis stage uses a system prompt that establishes three cognitive phases for the identification task:

1. **Direct visual analysis:** The model is instructed to examine the above- and below-view photographs carefully *before* reading any tool outputs, and to form its own preliminary diagnosis based solely on visual evidence.
2. **Critical evaluation of subsidiary evidence:** The model reviews the outputs from four assistant tools. The CNN classifier, the trait extractor, the identification-key traversal, and the trait-database validation step. Each tagged with explicit warnings about its failure modes (CNN overconfidence on out-of-distribution specimens, segmentation errors in computer-vision measurements, key-traversal sensitivity to trait precision, and database coarseness). The model is instructed to compare each tool output with its Phase 1 visual analysis and flag contradictions.
3. **Synthesis and final decision:** The model is forbidden from simply voting or averaging the tool outputs. It must use the injected identification key to test competing hypotheses when evidence conflicts, choose the species with the strongest overall evidence (even if this contradicts its preliminary diagnosis), and return "Unknown" when no species has strong evidence. A safety instruction requires conservative abstention when a toxic lookalike is possible.

The final synthesis prompt also includes a compact text representation of the XML identification key, generated from parsed root-to-leaf decision paths rather than by inserting the raw `key.xml` file. A compact species list, containing English and scientific names, is also included to constrain predictions. The model must return a structured JSON response containing its top prediction, alternative hypotheses, per-tool trust ratings, agreement state, and safety warnings.

The MLLM experiments used gemma3:12b through Ollama with a temperature of 0, a context length of 8192 tokens, and a maximum output length of 4096 tokens. No explicit seed or top-p value was configured, so these settings remained at their default values. Each MLLM call was submitted as a separate stateless chat request, without messages from previously evaluated specimens. Each standalone MLLM condition was executed once per specimen. Each unified B1 and B2 run invoked the MLLM twice per specimen: once for identification-key navigation and once for final evidence synthesis.

3.9.1 Input Construction

The current unified pipeline uses the MLLM in two separate stages: first for one-shot navigation of the Swedish identification key, and then for final synthesis of all evidence sources. Both stages receive structured text inputs, and both can also receive the above- and below-view photographs as base64-encoded JPEG images unless image input is disabled for an experiment.

LLM-assisted key navigation. For key navigation, the system prompt contains the full XML-derived identification key in a compact text representation. The user message contains the merged `visible_traits` dictionary, the CNN top prediction when available, and the detected morphology case with the detected parts. The model is instructed to choose a complete path through the identification key and return the sequence of question–answer pairs as JSON. The returned path is validated against `key.xml`. If the path is valid and reaches a leaf, the tree subsystem records the corresponding species as a key-traversal conclusion. If the path is invalid or incomplete, the pipeline keeps the deepest valid partial context and reports the unresolved question and available options to the final synthesis stage. If the LLM is not available, the pipeline falls back to the deterministic `KeyTreeEngine` auto-answer logic.

Final synthesis prompt. The final synthesis prompt asks the LLM to identify the mushroom from both the images and the structured subsystem outputs. The system prompt instructs the model to follow three steps: first inspect the images directly, then critically evaluate the tool outputs and their known failure modes, and finally synthesise the evidence without simply voting or averaging the tools. The identification key and supported species list are included in the system prompt as reference material, and the model is instructed to return JSON only.

The user message for final synthesis is built from the following evidence blocks:

- **Morphological case and detected parts:** the case detected from the YOLOv8 part masks, currently `classical`, `puffball`, or `uncertain`, together with the accepted part masks such as cap, stem, and underside.
- **Specimen images:** the above- and below-view images, passed to the MLLM as image inputs. These images allow the MLLM to form an independent visual judgement before reviewing the tool outputs.
- **CNN classifier report:** the CNN top prediction, confidence score, conclusive flag, uncertainty reason when available, and top-5 alternatives.
- **Identification-key report:** the validated key-traversal result. This may be a species conclusion, or an incomplete traversal with the question and options where the path stalled.
- **Trait-database report:** the database validation report for the key-traversal conclusion when available, otherwise for the CNN top species. If neither candidate is available, the database report is marked as skipped. In the final synthesis prompt, this report is represented as a compact summary containing the candidate species checked by the comparator, its trait-match score, the number of conflicting traits, and documented look-alikes when available. The full stored database result also contains the underlying matched traits, conflicts, non-comparable traits, candidate metadata, and safety-alert information.
- **Extracted traits reference:** the full merged `visible_traits` dictionary. This includes the part-aware traits, such as `cap_color`, `stem_color`, `underside_color`, `hymenophore_type`, `stem_ring`, `stem_surface`, and `morphology_case`, as well as derived legacy compatibility fields such as `dominant_color`, `secondary_color`, `surface_texture`, `has_ridges`, `brightness`, and `colour_ratios`. Trait-confidence is included so that the MLLM can distinguish strong masked evidence from uncertain observations.

In oracle-based benchmark variants, the same prompt structure is retained, but the extracted trait dictionary is replaced by oracle traits derived from the reference trait database. This allows the evaluation to isolate downstream reasoning performance from visual trait-extraction errors.

3.9.2 Fallback Behaviour

The pipeline also implements a fallback mechanism for cases where the final MLLM output is unusable, for example if it returns `Unknown`, `Error`, or cannot be parsed. In such cases, the final recommendation is selected from the available non-MLLM signals using a fixed priority order: a valid key-traversal conclusion, then a conclusive CNN prediction, and finally the trait-database candidate. This mechanism was included to make the pipeline robust to MLLM failure. However, it did not affect the Benchmark 5 results, as neither B1 nor B2 triggered fallback.

3.9.3 Response Parsing and Agreement Evaluation

Both LLM stages are requested to return JSON-formatted responses. Since model outputs may occasionally deviate from the requested format, the parser is implemented defensively. It first attempts direct JSON parsing, then removes common wrappers such as Markdown code fences, and finally searches for the largest valid JSON object within the response. For the final synthesis stage, the parser also accepts several equivalent field names for the predicted species and confidence score, so that minor formatting deviations do not automatically invalidate an otherwise usable response.

After the final synthesis stage, the pipeline computes its own agreement label across the available subsystem signals: the CNN prediction, the validated identification-key result, the database-validated candidate, and the LLM prediction. The resulting agreement state is one of *agree*,*partial*, *disagree*, or *inconclusive*. This agreement value is a deterministic post-processing result and is used in the final recommendation, even if the LLM also reports its own agreement assessment in the raw synthesis output. This separates the model's interpretation of the evidence from the pipeline's fixed comparison logic.

The final recommendation reports the selected species, confidence, toxicity metadata, safety warnings, clarification state, subsystem signals, fallback status, and the computed agreement label. The full benchmark output also stores the individual subsystem predictions separately, allowing later analysis of where the CNN, key traversal, database validation step, and LLM agreed or contradicted one another.

3.10 Evaluation Procedure

This thesis evaluates the identification pipeline through a comparative benchmark that measures the accuracy of each System A method and both variants of the System B pipeline on a held-out evaluation set. Table 1 summarises the benchmarked methods and input conditions.

Table 1: Benchmarked identification methods and input conditions

Method	System	Input	Description
CNN (`cnn`)	A	Single photo, best detection	EfficientNet-B3 classifier with softmax output.
A1 image-only LLM (`a1_vision`)	A1	Above and below photos	Standalone vision LLM using only the image pair, with no auxiliary trait data.
A1 image and extracted-traits LLM (`a1_llm`)	A1	Above and below photos with extracted traits	Standalone vision LLM using the image pair together with the merged `visible_traits` output from the trait extractor.
A1 key traversal (`a1_tree`)	A1	Extracted traits	Deterministic identification-key traversal using automatically extracted traits.
A1 trait-database comparator (`a1_db`)	A1	Extracted traits	Standalone full-database ranking against `species_traits.xml`.
A2 image and oracle-traits LLM (`a2_llm`)	A2	Above and below photos with oracle traits	Standalone vision LLM using the image pair together with ground-truth oracle traits.
A2 key traversal (`a2_tree`)	A2	Oracle traits	Deterministic identification-key traversal using oracle traits.
A2 trait-database comparator (`a2_db`)	A2	Oracle traits	Standalone full-database ranking against `species_traits.xml` using oracle traits.
B1 unified pipeline (`b1`)	B1	Above and below photos with extracted traits and subsystem evidence	Unified MLLM synthesis pipeline using automatically extracted traits, CNN evidence, key traversal, and trait-database validation.
B2 unified pipeline (`b2`)	B2	Above and below photos with oracle traits and subsystem evidence	Unified MLLM synthesis pipeline using oracle traits, CNN evidence, key traversal, and trait-database validation.

3.10.1 Benchmark Evaluation Procedure

The benchmark design introduced in Section 3.10 is used to compare the standalone identification methods and the unified pipeline variants on the same held-out image set. Its purpose is to measure top-1 identification accuracy, coverage, and inference time under matched input conditions, and to separate errors caused by automatic trait extraction from errors caused by downstream reasoning.

The comparative benchmark reads an evaluation manifest from a comma-separated value (CSV) file. The manifest contains the columns `specimen_id`, `above_image_path`, `below_image_path`, `species_id`, `scenario`, `subset`,

`confusing_pair_with`, and `notes`. The two image-path columns refer to the cap-view and underside-view photographs of the same specimen. The `species_id` column provides the ground-truth species identifier. The `scenario` column groups specimens into the benchmark categories `standard`, `confusing`, and `ood`, where `ood` denotes out-of-distribution cases.

For each specimen, every benchmarked method returns a `RunnerResult`. This common result object contains a ranked prediction list, a coverage flag, run time, optional error information, and method-specific metadata. The top predicted species and its confidence score are taken from the first entry in the ranked prediction list. The confidence score is therefore method-specific: for the convolutional neural network it is the top softmax score, while for the language-model methods it is the confidence value returned by the model and parsed from the JSON response.

The benchmark computes the following per-method metrics:

- **Top-1 accuracy:** the proportion of covered predictions for which the top predicted species matches the ground-truth `species_id`. Methods that abstain or fail to return a usable prediction are excluded from the accuracy denominator and are instead reflected in the coverage metric.
- **Coverage:** the proportion of specimens for which the method returns a usable prediction rather than an abstention, an empty prediction, or an error.
- **Mean inference time:** the average wall-clock inference time per specimen, reported in milliseconds.

Because top-1 accuracy is computed over covered predictions, it must be interpreted together with coverage. A method with low coverage may achieve high accuracy on the few specimens it predicts while still being unsuitable as a complete identification system.

The benchmark also computes comparative metrics across methods:

- **Scenario breakdown:** the per-method accuracy for the `standard`, `confusing`, and `ood` scenario groups.
- **Confusing-pair breakdown:** the per-method accuracy on documented lookalike pairs identified by `confusing_pair_with`.
- **Oracle benefit:** the accuracy difference between oracle-trait and extracted-trait variants for the standalone language model, key traversal, and trait-database comparator.
- **Synthesis benefit:** the accuracy difference between each unified pipeline variant and the best standalone method in the corresponding benchmark configuration.
- **Trait-extractor impact:** the accuracy difference between B1 and B2, comparing the unified pipeline with extracted traits against the same pipeline with oracle traits.

No randomisation is used by the comparative benchmark runner itself. Each specimen is therefore evaluated once per method in a fixed benchmark run, and the reported statistics are aggregate accuracy, coverage, and mean inference time rather than repeated-run confidence intervals. The software setup for a benchmark run is defined by the project version, the benchmark command, the trained CNN and YOLOv8 checkpoints, and the local LLM backend used for inference. These details are necessary because changing any of them can change the benchmark output. The hardware platform should be reported with the result tables because inference time can change depending on the CPU, GPU, memory, and local LLM runtime available on the evaluation machine.

The benchmark writes three main report artefacts for each run: a structured JSON report, a CSV report, and a Markdown summary. The JSON report preserves the per-specimen method outputs, including each method's prediction, confidence, coverage, correctness, reasoning when available, and, for unified-pipeline runs, the subsystem signals used by B1 and B2. These include the CNN prediction, key-traversal result, database-checked candidate, language-model prediction, fallback status, and agreement label.

The CSV report contains one row per specimen and records each method's prediction, correctness, coverage, and confidence. For B1 and B2, it also stores the raw LLM prediction, LLM confidence, CNN prediction, key-traversal result, database-checked candidate, database species identifier, and whether fallback was used. The Markdown summary presents the aggregate benchmark metrics in tables for reporting and inspection. In addition to these reports, the run also stores a `checkpoint.json` file, which preserves fuller runner metadata for later inspection or resumed execution.

The benchmark results are made traceable through the implementation artefacts and generated result files. The source code, benchmark scripts, configuration files, and model checkpoints define which components were executed, while the JSON, CSV, and Markdown reports store the outputs of the benchmark run. This allows the numerical tables to be traced back to a concrete execution on the held-out image set.

4 Results

This section reports the comparative benchmark described in Section 3.10. The run used 30 held-out specimens: 10 standard identification cases, 10 confusing lookalike cases, and 10 out-of-distribution cases. Each specimen was evaluated once by each benchmarked method. The benchmark artefacts were written to `benchmarks/Results/5/extract` and contain a structured JavaScript Object Notation (JSON) report, a comma-separated values (CSV) report, and a Markdown summary. The saved run was completed on 26 May 2026.

The reported accuracy values follow the benchmark definition from Section 3.10.1. Accuracy is computed over covered predictions only. For methods with full coverage, this is the same as accuracy over all 30 specimens. For low-coverage methods, such as key traversal, the accuracy value must therefore be read together with coverage. The results chapter reports the measured benchmark outcomes; explanations and limitations are discussed in Section 5.

4.1 Overall Benchmark Performance

Table 2 reports the overall benchmark performance. The convolutional neural network (CNN) produced the strongest full-coverage single-photo baseline using the higher-confidence result from the two specimen views, with 56.7% accuracy. Among the unified-pipeline variants, B1 reached 50.0% accuracy and B2 reached 53.3% accuracy. The highest covered accuracy in the extracted-trait configuration was observed for `a1_tree` at 75.0%. However, this value was based on only four covered specimens and should therefore not be interpreted as full-system performance. The strongest full-coverage oracle-trait standalone result was the A2 database comparator, with 73.3% accuracy.

Table 2: Overall Benchmark 5 results. Accuracy is top-1 accuracy over covered predictions.

Method	Correct	Covered	Accuracy	Coverage	Mean Time (ms)
CNN	17	30	56.7%	100.0%	697.5
`a1_vision`	7	30	23.3%	100.0%	18385.1
`a1_llm`	8	30	26.7%	100.0%	15547.5
`a1_tree`	3	4	75.0%	13.3%	1425.9
`a1_db`	2	30	6.7%	100.0%	1274.4
`a2_llm`	11	30	36.7%	100.0%	15893.6
`a2_tree`	3	6	50.0%	20.0%	112.6
`a2_db`	22	30	73.3%	100.0%	5.9
B1	15	30	50.0%	100.0%	85315.3
B2	16	30	53.3%	100.0%	84925.1

The run-time values in Table 2 are local wall-clock measurements from the benchmark run, reported in milliseconds. They are useful for comparing methods within this run, but they should not be interpreted as hardware-independent performance claims. The language-model methods and the unified-pipeline variants were slower than the CNN and database comparator because they depend on local multimodal language-model inference through Ollama.

4.2 Comparative Accuracy Differences

Table 3 summarises the main cross-method comparisons. The oracle-trait LLM improved over the extracted-trait LLM by 10.0 percentage points. The database comparator showed the largest measured oracle-trait difference, increasing from 6.7% with extracted traits to 73.3% with oracle traits. This difference suggests that the database comparator was sensitive to the quality of the input traits.

In the full-coverage comparison, B1 was 6.7 percentage points below the CNN baseline. B2 was 3.3 percentage points higher than B1, which shows a small gain when the unified pipeline received oracle traits instead of automatically extracted traits.

Table 3: Benchmark comparative accuracy differences for full-coverage methods. Values are percentage-point differences.

Comparison	Baseline	Comparison Method	Difference
LLM oracle benefit: `a2_llm` versus `a1_llm`	26.7%	36.7%	+10.0
Database oracle benefit: `a2_db` versus `a1_db`	6.7%	73.3%	+66.7
Unified oracle gain: B2 versus B1	50.0%	53.3%	+3.3
B1 versus CNN baseline	56.7%	50.0%	-6.7

4.3 Scenario-Level Results

Table 4 reports the scenario-level accuracy for the full-coverage methods. The CNN performed best on the standard identification group (80.0%) and remained strong on the confusing group (70.0%), but dropped to 20.0% on out-of-distribution specimens. B1 and B2 followed a similar pattern. Both variants performed strongest on confusing specimens and weakest on out-of-distribution specimens. B2 improved over B1 on the standard identification group, reaching 70.0% instead of 60.0%, but both variants reached only 10.0% on out-of-distribution cases.

Table 4: Benchmark accuracy by scenario for full-coverage methods. Each scenario contains 10 specimens.

Scenario	CNN	`a1_vision`	`a1_llm`	`a1_db`	`a2_llm`	`a2_db`	B1	B2
`standard`	80.0%	20.0%	40.0%	20.0%	20.0%	100.0%	60.0%	70.0%
`confusing`	70.0%	40.0%	20.0%	0.0%	70.0%	40.0%	80.0%	80.0%
`ood`	20.0%	10.0%	20.0%	0.0%	20.0%	80.0%	10.0%	10.0%

The out-of-distribution results show that the unified pipeline did not compensate for specimens outside the main benchmark species distribution in this run. Despite receiving multiple evidence sources, the unified pipeline remained weak on the out-of-distribution group. This scenario should be interpreted as an OOD-labelled benchmark group rather than a pure outside-label-space evaluation, because at least one OOD-labelled species was still present in the CNN output space.

4.4 Low-Coverage Identification-Key Results

The identification-key methods are reported separately because their coverage was substantially lower than the other methods. In the extracted-trait condition, `a1_tree` produced predictions for 4 of 30 specimens and was correct for 3 of those covered predictions. In the oracle-trait condition, `a2_tree` produced predictions for 6 of 30 specimens and was correct for 3 of those covered predictions. These results show that deterministic key traversal could be accurate when it reached a conclusion, but it did not provide sufficient coverage to function as a full benchmark method in this run.

The specimens for which the standalone key methods produced conclusions are listed in Table 11 in Appendix C. The appendix table distinguishes between conclusions reached by full XML traversal and conclusions returned by the precheck mechanism described in Section 3.6. In Benchmark 5, precheck conclusions occurred for `AM.VI_007` (Destroying Angel) and `HY.PS_019` (False Chanterelle). The benchmark contained one Destroying Angel specimen and two False Chanterelle specimens, but only one of the False Chanterelle specimens reached a precheck conclusion.

In the unified B1 and B2 runs, the LLM-based key-navigation step did not yield any validated species-level key conclusions in Benchmark 5. The stored `b1_tree` and `b2_tree` fields were empty for all 30 specimens. No proposed LLM key path was accepted by the deterministic `TreePathValidator` as a complete valid root-to-leaf traversal.

4.5 Confusing Lookalike Results

The confusing subset contained ten specimens. Two four-specimen lookalike groups were represented: Bay Bolete–Porcini and Chanterelle–False Chanterelle. For both groups, the unified B1 and B2 pipelines reached 75.0% accuracy, corresponding to three correct predictions out of four specimens in each group. The benchmark summary also contained two single-specimen confusing rows, Fly Agaric and Destroying Angel, for which the manifest did not specify a paired counterpart. These rows are included in the appendix for completeness, but they should not be generalised as pair-level lookalike results.

Across all ten specimens in the confusing scenario, B1 and B2 each reached 80.0% accuracy. This was higher than the CNN classifier at 70.0% and matched or exceeded the standalone language-model variants: the image-only LLM reached 40.0%, the extracted-trait LLM reached 20.0%, and the oracle-trait LLM reached 70.0%. The full method-by-method confusing-group breakdown is reported in Table 10 in Appendix C.

4.6 Qualitative Segmentation Errors

Manual inspection of the YOLOv8 part-segmentation overlays showed several failure modes that could affect downstream trait extraction. One of these issues was unreliable underside detection: in some images the underside was missed, while in others it was confused with the cap. The model also occasionally assigned mushroom-part labels to background objects, for example detecting a tree-like structure as a stem. Clustered specimens were another difficult case, because several fruiting bodies could be present in the same image while the trait extractor expects one dominant specimen. In one puffball-like case (`LY.PE_037`), the model labelled most of the mushroom body as underside, which caused the downstream trait extractor to analyse the wrong anatomical region. These observations show that segmentation errors can propagate into the extracted traits.

Table 5: Examples of qualitative YOLOv8 segmentation errors observed in Benchmark 5.

Error Type	Example	Effect on Trait Extraction
Missed underside	Multiple underside-view images	`hymenophore_type` becomes unknown.
Background as stem	Tree or background object detected as stem	Stem colour and texture become unreliable.
Underside as cap	Underside region labelled as cap	Cap colour or shape is measured from the wrong part.
Clustered mushrooms	Multiple fruiting bodies in one image	Traits may mix several individuals.
Whole body as underside	`LY.PE_037`	Puffball or body traits are replaced by underside evidence.

4.7 Fallback and Agreement Metadata

The fallback mechanism described in Section 3.9.2 was not used in Benchmark 5. The JSON and CSV reports record fallback status for B1 and B2, and both variants had 0 fallback-triggered specimens out of 30. This means that all B1 and B2 final recommendations in this run came from the final language-model synthesis stage rather than from the deterministic fallback order.

The unified-pipeline JSON metadata also stores an agreement label for each specimen. For both B1 and B2, the distribution was 26 *agree*, 2 *partial*, and 2 *disagree*. Agreement here means agreement among available component predictions, not agreement with the ground truth. In the saved run, B1 was correct for 14 of the 26 specimens labelled *agree*, while B2 was also correct for 14 of the 26 specimens labelled *agree*. Agreement therefore indicates consistency among component outputs, not correctness of the final identification.

4.8 Example Per-Specimen Output

Table 6 shows one per-specimen B2 output from the JSON report. The specimen `AG.AU_001` has ground-truth species `AG.AU`, but B2 predicted `MA.PR`. The component signals did not all agree: the CNN and database validation both pointed to `LA.HE`, while the final language-model synthesis selected `MA.PR`. The computed agreement label was therefore *partial*, and fallback was not used.

This example illustrates why the benchmark stores component-level metadata in addition to the final prediction. The final output alone shows that the specimen was misidentified, while the component metadata shows which subsidiary signals supported or contradicted the final recommendation.

4.9 Complementarity Between CNN and Unified Pipeline Predictions

Table 7 compares whether the CNN and unified pipelines were correct on the same specimens.

The union value is an upper-bound overlap analysis and was not produced by an implemented selection mechanism. Although the unified systems did not outperform the CNN in aggregate accuracy, their errors were not identical to the CNN errors. B1 correctly identified two specimens that the CNN misclassified, while the CNN correctly identified

Table 6: Example B2 per-specimen output from Benchmark 5.

Field	Value
Specimen	`AG.AU_001`
Ground truth	`AG.AU` (*Agaricus augustus*)
B2 prediction	`MA.PR` (*Macrolepiota procera*)
B2 confidence	0.78
Correct	no
CNN signal	*Lakrits Milkcap*, confidence 0.6389
Key-traversal signal	no conclusion
Database signal	`LA.HE` (*Lakrits Milkcap*), score 0.419
Fallback used	no
Agreement label	*partial*

Table 7: Overlap between CNN and unified-pipeline correct predictions

Comparison	Both correct	CNN only	Unified only	Both wrong	Union correct
CNN vs B1	13	4	2	11	19/30
CNN vs B2	14	3	2	11	19/30

four specimens missed by B1. Similarly, B2 correctly identified two CNN misses, while the CNN correctly identified three B2 misses. The union of correct predictions between CNN and either unified system was therefore 19 out of 30 specimens (63.3%), compared with 17 out of 30 for the CNN alone, 15 out of 30 for B1, and 16 out of 30 for B2.

4.10 Summary of Results

In summary, Benchmark 5 did not show an overall accuracy advantage for the unified pipeline over the strongest standalone methods. B2 improved slightly over B1, indicating a small measured benefit from oracle traits in the unified setting, but both unified variants performed poorly on out-of-distribution specimens. The strongest full-coverage standalone result was the oracle-trait database comparator, while the strongest full-coverage image-only baseline was the CNN. However, the CNN and unified systems made partly different errors: the union of correct CNN and unified-pipeline predictions reached 19 of 30 specimens in both the B1 and B2 comparisons.

5 Discussion

This chapter interprets the Benchmark 5 results in relation to the main contributions of this thesis. The thesis contributes a part-aware mushroom trait extractor, an implementation of trait-based identification-key traversal, a trait-database comparison module, a multimodal LLM synthesis pipeline, and a benchmark framework that compares the unified pipeline with its standalone components. The discussion focuses on what the benchmark results show about accuracy, robustness, and limitations, and on how the current implementation could be improved.

5.1 Answering the Research Question

The main research question was: *How does combining machine learning, trait-based identification, and large language model classification affect mushroom identification compared with using each method as a standalone approach?*

The results show that combining the components did not improve overall top-1 accuracy compared with the strongest standalone methods. As shown in Table 2, neither unified variant exceeded the CNN baseline or the oracle-trait database comparator. B2 improved only slightly over B1, suggesting that replacing automatically extracted traits with oracle traits produced only a limited gain in the unified setting. The direct answer to the research question is therefore that the current unified pipeline provided evidence integration, but did not produce a general accuracy advantage over the strongest standalone components. On the confusing-lookalike subset, however, both unified variants reached 80.0% accuracy, exceeding the CNN's 70.0%, and the union of correct CNN and unified predictions covered 19 of 30 specimens, indicating that the errors were partly complementary.

The result is therefore mixed: the architecture demonstrated potential as an evidence-integration framework, however current synthesis strategy and supporting components were not reliable enough to make the unified prediction consistently stronger than the best standalone method.

5.2 Benchmark Performance

5.2.1 Standalone Performance

The standalone results show that different components had different strengths. The CNN was the strongest full-coverage image-based baseline. In the standalone benchmark, the CNN was applied to both the above-view and below-view images when both were available, and the prediction with the higher top confidence was kept. This gave the standalone CNN a direct opportunity to use the more informative of the two views. The result suggests that the image classifier captured useful visual regularities for the species included in its label space. In contrast, the standalone MLLM conditions were weaker than the CNN, indicating that the local multimodal language model was not a stronger standalone visual classifier than the trained CNN in this benchmark.

The strongest full-coverage structured result was the oracle-trait database comparator. Its performance was much higher with oracle traits than with automatically extracted traits, as shown in Table 3. This suggests that the database-comparison mechanism can be useful when supplied with reliable trait information, but that its performance depends strongly on the quality and completeness of the input traits. Since the comparator performed well with oracle traits, its poor performance with automatically extracted traits is consistent with errors or omissions in the extracted trait evidence rather than limitations in the database-matching logic alone.

The standalone key traversal was different from the other full-coverage methods because it produced conclusions for only a small number of specimens. Its accuracy therefore has to be interpreted together with coverage. When the key reached a conclusion, it could be correct, but the low coverage means it was not sufficient as a complete independent identification method in this benchmark.

The results in Table 2 also show that the image-only MLLM performed worse than the MLLM condition that received the same images together with oracle traits. This suggests that the MLLM could benefit from additional relevant trait context when the context was directly related to the specimen and presented in a focused prompt.

5.2.2 Unified Pipeline Performance

The scenario-level results in Table 4 show where the unified pipeline helped and where it failed. Benchmark 5 contained standard cases, confusing-lookalike cases, and out-of-distribution cases. The unified systems were competitive in the confusing-lookalike group, where B1 and B2 both exceeded the CNN. This suggests that the unified pipeline may be useful in some visually ambiguous cases, although the scenario-level results alone do not show whether the improvement came from trait evidence, database comparison, MLLM reasoning, or their combination.

Overall the unified systems did not preserve the strongest advantages of all their supporting components. The oracle-trait database comparator performed well as a standalone full-database ranking method, but this result is not directly equivalent to the database signal used inside B2. In the unified pipeline, the database component checks a candidate proposed by the key traversal, or by the CNN when no key conclusion is available, rather than ranking all species independently. B2 therefore did not receive the same kind of standalone full-database prediction as `a2_db`. This means that the lower B2 result should not be interpreted simply as the MLLM ignoring the strongest structured trait match, but also as a consequence of how the database component was used in the unified pipeline.

The unified systems also remained weak on the OOD group. This indicates that providing additional evidence sources did not, by itself, make the system robust to distribution shift. The database validation design may have contributed to this result, However this does not fully explain the result. On the OOD subset, B1 and B2 performed below the simpler standalone MLLM conditions. One possible explanation is that the larger final-synthesis prompt made it harder for the model to prioritise the most relevant evidence. This interpretation is consistent with the long-context limitations discussed in Section 2.8, where additional context may increase the amount of information the model must reconcile rather than automatically improving prediction quality. The results are therefore consistent with the local Gemma 3 12B model performing better with focused image-and-trait prompts than with the broader unified synthesis prompt, although prompt structure was not evaluated independently.

5.2.3 Complementary Errors Between CNN and Unified Systems

The overlap analysis in Table 7 shows that the unified pipeline did not simply reproduce the CNN's error pattern. The CNN and unified systems made partly complementary errors: the union of correct CNN and unified-pipeline predictions reached 19 of 30 specimens, compared with 17 correct predictions for the CNN alone. This union is not an achieved system result, because no implemented selection mechanism chose between the CNN and unified outputs. However, it suggests that the evidence sources contain complementary information that the current synthesis strategy did not fully exploit.

This distinction is important for interpreting the research question. The unified pipeline did not improve overall benchmark accuracy, but it did recover from some CNN-specific failures. Future versions may therefore benefit less from simply adding more evidence and more from improving the mechanism that decides which evidence source should be trusted when the components disagree.

5.3 MLLM Synthesis Limitations

The limited difference between B1 and B2 suggests that the MLLM synthesis stage did not fully benefit from oracle traits. This indicates that the limitation was not only the quality of the extracted traits, but also how the language model integrated the available evidence. In the unified pipeline, the model had to interpret visual information, trait descriptions, subsystem outputs, and candidate species information within a single synthesis step. The oracle-trait prompt supplied more complete species-level information, but not necessarily evidence that was specific to the photographed specimen or easy for the model to prioritise. As a result, the additional traits may have increased the amount of evidence the model had to reconcile rather than clearly reducing ambiguity in the decision-making process.

Execution pattern. This also suggests that the execution pattern of the MLLM component may have affected the result. In the unified pipeline, the model was not only asked to recognise visual features, but also to reconcile extracted or oracle traits, subsystem outputs, candidate species, and key information within a final synthesis prompt. The limited gain from oracle traits therefore does not necessarily show that trait information is unhelpful for language-model reasoning. Rather, it suggests that providing more evidence in one large prompt may be a brittle way to use the model, especially when the evidence sources must be prioritised or resolved against one another.

This interpretation is consistent with the evidence-fusion background discussed in Section 2.8. Adding more information to an LLM prompt does not guarantee a better prediction, especially when the prompt becomes long, redundant, or contains evidence that must be weighed against visual impressions. One plausible interpretation is that the MLLM balanced the supplied traits against its own visual interpretation and the other subsystem outputs rather than treating the trait evidence as authoritative.

The results also reveal a difference between structured computation and language-model-based synthesis. The standalone database comparator improved substantially when supplied with oracle traits because it processed those traits through a fixed matching procedure. The MLLM, by contrast, had to interpret the structured traits together with images and outputs from the other subsystems. Furthermore, the database comparator served a different role within the unified system. Rather than ranking all species profiles independently, it evaluated a candidate species supplied by the identification key or CNN. This methodological difference may partly explain the performance gap

between the standalone database comparator in the A benchmark and the database-supported synthesis used in the unified systems.

A further limitation concerns key navigation in the unified B-family systems. The MLLM-based key-navigation step did not produce any accepted species-level key conclusions in Benchmark 5. No proposed LLM key path was accepted by `TreePathValidator` as a complete valid root-to-leaf traversal. This suggests that asking the model to produce a full valid traversal path is a brittle way to use the key, especially when the key is long and the model must choose among many branch options.

5.4 Key Traversal and Interpretability

The key-traversal results show both the value and the limitations of explicit trait-based reasoning. The deterministic standalone key traversal had low coverage in Benchmark 5. With extracted traits, `a1_tree` reached a conclusion for four specimens, while with oracle traits, `a2_tree` reached a conclusion for six specimens. This low coverage partly reflects the scope of the XML key: not all benchmark species were represented as reachable leaf decisions, and some species appeared only as lookalikes. It also reflects that several key questions required traits that were missing from the available evidence.

However, the key remained useful from an interpretability perspective. Unlike a CNN softmax prediction, a key traversal can expose the sequence of character decisions that led to a conclusion or show where traversal stopped. This is valuable in mushroom identification because it makes the reasoning path inspectable. It also clarifies when the available traits are insufficient for a field-guide-style identification.

At the same time, the standalone key traversal was not completely independent from the CNN classifier. The key engine accepts a CNN `ml_hint`, consisting of the CNN's top predicted species and confidence score. For a small number of predefined species or question pairs, this hint can be mapped to likely key answers. In some cases, the engine can also return a precheck conclusion for species that are not represented as normal traversable leaves. This design can be useful in an interactive system, where image classification is allowed to guide the key. In the benchmark, however, it weakens the interpretation of `a1_tree` and `a2_tree` as purely trait-based baselines.

The incorrect Black Trumpet conclusion for the Oyster Mushroom specimen `PL.OS_045` illustrates this issue. The CNN predicted Black Trumpet for this specimen with high confidence. Because the key engine used the CNN hint to answer some key questions, the traversal was guided toward the XML leaf for Black Trumpet. This result should therefore not be interpreted as an independent trait-based confirmation. The oracle traits for Oyster Mushroom instead describe a gilled hymenophore rather than ridges, suggesting that the error was introduced by propagating the incorrect CNN hint through the key traversal.

These results suggest that key traversal is valuable as an interpretable component, but that its current implementation is not sufficient as a high-coverage standalone identifier.

5.5 Trait Extraction as a Bottleneck

The clearest evidence that trait extraction was a bottleneck is the difference between the extracted-trait and oracle-trait database results. The database comparator performed poorly with automatically extracted traits but strongly with oracle traits. This suggests that the structured database-comparison mechanism can work when supplied with reliable traits, while the automatic visual trait extractor often failed to provide sufficiently accurate or complete evidence.

Segmentation dependency. The trait extractor depends on the correctness of the YOLOv8 part labels and segmentation masks. The qualitative segmentation errors reported in Table 5 show that the model sometimes assigned mushroom-part labels to incorrect image regions, confused underside with cap, or labelled most of a compact fruiting body as underside. Since the visual trait extractor measures colour, shape, texture, and hymenophore structure inside these masks, an incorrect mask causes the downstream traits to describe the wrong object or anatomical part.

Instance merging. The trait extraction pipeline is designed around one dominant fruiting body. When several detections of the same part class are present, the mask-building step keeps the highest-confidence mask and may merge nearby or overlapping masks of the same class. This is reasonable for images containing one clear mushroom, but it becomes problematic for clustered specimens. In such cases, the selected cap, stem, and underside may come from different individuals, or several caps may be merged into one irregular mask. This reduces the reliability of measurements such as colour clustering, contour shape, edge density, and hymenophore detection, because the extracted traits no longer describe a single coherent specimen.

Heuristic rules. Another limitation is the extractor's reliance on manually defined thresholds. Many traits are derived from heuristic rules, such as edge density for surface texture, contour measurements for cap shape, and line or blob detection for hymenophore type. As discussed in Section 2.7, such classical image-processing measurements provide useful approximate cues, but they are sensitive to lighting, viewpoint, occlusion, image sharpness, background clutter, and natural morphological variation. These limitations help explain why extracted traits did not provide the same benefit as oracle traits. Furthermore, the heuristic rules are highly sensitive to the quality of the YOLOv8 segmentation masks, because the masks determine which image regions are analysed.

Traits available from images. The extractor is also limited by the kinds of traits that can be observed from the available images. It focuses on visible morphology, such as cap colour, cap shape, surface texture, stem appearance, and hymenophore type. However, as discussed in Section 2.1, mushroom identification often depends on characters that are not visible in two photographs, including smell, bruising reactions, latex, spore print colour, substrate, habitat, season, microscopic characters, and details of the stem base or gill attachment. These traits cannot be reliably recovered by the current image-based extractor.

Legacy and part-aware traits. The current trait extractor exports both legacy compatibility fields and newer part-aware traits. Legacy fields such as `dominant_color`, `secondary_color`, `surface_texture`, and `has_ridges` collapse information across the mushroom or represent a specific anatomical feature as a coarse value. In contrast, part-aware fields such as `cap_color`, `stem_color`, `underside_color`, `hymenophore_type`, `cap_surface`, `stem_surface`, and `stem_ring` retain which anatomical part the observation belongs to. Relying more directly on the part-aware fields could improve downstream reasoning because the key and database comparator often require part-specific evidence rather than whole-mushroom summaries.

Trait mismatch with the database and key. The visual trait extractor produces a single `visible_traits` representation that is consumed by both the trait-database comparator and the identification-key traversal. However, these components require different forms of evidence. The database comparator performs weighted matching against species-level trait profiles, while the identification key requires answers to specific diagnostic questions from the XML field guide. Some of these questions can be approximated from the extracted traits, but many are not directly represented in the current extractor output. As a result, the same extracted traits may be sufficient for a partial database comparison but insufficient for automatic key traversal.

This mismatch helps explain why the oracle-trait database comparator achieved much higher accuracy than the extracted-trait comparator, while the key traversal still had low coverage even when oracle traits were supplied. For example, `hymenophore_type` can help answer the first key question about whether the underside has gills, pores, ridges, or teeth. However, later questions such as tube detachability or cap–stem attachment are not measured by the visual extractor, causing traversal to stop even when earlier branches are correct.

5.6 Benchmark, Dataset, and Comparability Limitations

One limitation of Benchmark 5 is that not all images are equally suitable for the current system design. Several images contain multiple fruiting bodies, while the current pipeline assumes one dominant target specimen. The segmentation and trait-extraction stages do not explicitly associate caps, stems, and undersides belonging to the same individual mushroom. As a result, clustered specimens can produce mixed or incomplete trait evidence.

There are also comparability limitations between methods. Some standalone components, especially the identification key, have much lower coverage than the unified pipeline. Accuracy values for these methods therefore describe performance only on the specimens where a prediction was produced. This makes direct comparison with full-coverage methods such as the CNN, database comparator, and B1/B2 pipelines difficult.

The OOD subset should also be interpreted carefully. It is not a pure outside-label-space evaluation, because at least one OOD-labelled species is still present in the CNN output space.

Benchmark 5 is also limited by its small size. Since the benchmark contains only 30 specimens, one additional correct or incorrect prediction corresponds to 3.3 percentage points. Small differences between methods, such as the difference between B1 and B2, should therefore be interpreted cautiously.

5.7 Future Work

Heuristics. Several improvements could be made to the current system. One direction is to replace or complement some of the heuristic trait-extraction rules with learned trait classifiers. A deep learning-based trait model trained

on annotated morphological traits could potentially handle a wider range of lighting, viewpoint, age, and texture variation than the current threshold-based rules.

Segmentation. A second improvement concerns segmentation. Future work could compare the YOLOv8-nano segmentation model with larger YOLO variants or with alternative instance-segmentation architectures, such as two-stage or transformer-based models. More importantly, the segmentation training data should be expanded with additional annotated cap, stem, and underside masks, including difficult cases such as partial occlusion, forest backgrounds, and background objects that should not be classified as mushroom parts.

Identification key. Future evaluations from the identification key should separate trait-derived key answers from CNN-guided answers so that the independent value of the key traversal can be measured more clearly.

LLM orchestration and prompt structure. Future work could investigate alternative ways of executing the MLLM component. In this thesis, the unified pipeline used two MLLM stages: one prompt for identification-key navigation and one prompt for final evidence synthesis. Although this separated key navigation from final synthesis, each stage still required the model to solve a complex task in one invocation. In particular, the final synthesis prompt contained images, extracted or oracle traits, CNN output, key-navigation results, database-validation results, and final synthesis instructions. This gave the model access to several evidence sources, but may also have made the reasoning task harder by increasing the amount of information the model had to prioritise.

A more controlled approach would be to divide the reasoning process into smaller stages, such as visual diagnosis, uncertain-trait extraction, candidate comparison, conflict explanation, and final synthesis. This could make the role of each evidence source easier to inspect and reduce the risk that useful evidence is lost inside a large synthesis prompt.

Future work could also compare the locally hosted MLLM with stronger multimodal models accessed through external APIs. Such models may offer improved visual reasoning, larger context windows, and more reliable structured output, although these advantages would need to be evaluated empirically.

MLLM traversal of structured identification keys. Future work could investigate the MLLM's ability to navigate structured identification keys as a separate task. In the current unified pipeline, the model received a compact representation derived from the XML key and was asked to generate a complete traversal path in one invocation. Although this functionality was used in B1 and B2, it was not evaluated as an isolated condition, and no MLLM-generated path produced an accepted species-level conclusion in Benchmark 5.

A dedicated evaluation could measure whether the model selects valid branches, remains consistent with the supplied traits, and reaches the correct species-level terminal decision. It could also compare complete-path generation with a constrained approach in which the model answers one key question at a time while the traversal state is maintained deterministically. Such an evaluation would be relevant not only to mushroom identification, but also to other multimodal tasks that require language models to navigate hierarchical decision structures.

Benchmark. Future benchmark versions should include an additional condition in which the MLLM is supplied only with oracle traits, without photographs or outputs from the other identification methods. This condition would evaluate how well the model can map structured morphological information to a species prediction under idealised input conditions. It would separate trait-to-species reasoning from the more complex task of reconciling potentially conflicting evidence from the CNN, identification key, trait database, and images. However, the result would reflect both the model's mushroom knowledge and its ability to reason from the supplied traits, rather than reasoning ability alone.

The current oracle traits are species-level reference traits retrieved from `species_traits.xml` using the ground-truth species label. They may therefore include characteristics that are not visible in the benchmark photographs. A further benchmark condition should use manually verified, specimen-level oracle traits restricted to characteristics that can actually be observed in the supplied images. Comparing these conditions would help distinguish errors caused by imperfect automatic trait extraction from errors caused by missing visual evidence or limitations in the MLLM's trait-based identification.

Dataset. Finally, future benchmark runs should use a larger and more balanced held-out dataset. The benchmark should include repeated runs, clearer out-of-distribution groups, saved segmentation diagnostics, and explicit logging of key traversal depth, stuck nodes, and whether a conclusion came from an XML leaf or a precheck. This would

make it easier to separate failures caused by segmentation, trait extraction, key traversal, database matching, and LLM synthesis.

6 Conclusions

This thesis set out to investigate the following research question: *How does combining machine learning, trait-based identification, and multimodal large language model synthesis affect mushroom identification compared with using each method as a standalone approach?*

To answer this question, the thesis implemented and evaluated a unified mushroom-identification pipeline consisting of several components: an EfficientNet-B3 CNN species classifier trained with transfer learning, a YOLOv8-based part segmentation and visual trait extraction stage, a trait-database comparator, an XML-based identification-key traversal engine, and a multimodal LLM synthesis stage. These components were evaluated in Benchmark 5 through standalone baselines and two unified variants. B1 used automatically extracted traits, while B2 replaced these traits with oracle traits in order to separate trait-extraction errors from downstream reasoning errors.

The benchmark results show that the unified pipeline did not outperform the strongest standalone methods overall. B1 reached 50.0% top-1 accuracy and B2 reached 53.3%, compared with 56.7% for the CNN classifier and 73.3% for the oracle-trait database comparator. The CNN was the strongest practical full-coverage image-based baseline, while the oracle-trait database comparator showed that structured trait matching can be effective when supplied with reliable trait information. These results suggest that simply combining multiple sources of evidence does not guarantee improved identification accuracy. The quality of the individual components, especially segmentation, trait extraction, key traversal, and synthesis, strongly affects the final result.

However, the unified approach did show value in selected cases. B2 corrected some CNN errors, indicating that the evidence sources contained complementary information. At the same time, B2 also failed on some specimens that the CNN classified correctly. The main conclusion is therefore that the unified design is promising as an evidence-integration framework, but that the current implementation did not exploit the available evidence reliably enough to consistently outperform the best standalone component.

The thesis contributes a working benchmark framework and an implemented prototype that connects learned image classification, part-aware trait extraction, trait-based database comparison, identification-key traversal, and LLM-based synthesis. The results also identify the main bottlenecks for future work: improving segmentation quality, replacing brittle heuristic trait rules with learned or better-validated trait models, aligning the trait representation more closely with the database and key, improving key-traversal coverage, and making the LLM synthesis stage more transparent and reliable. Given the safety-critical nature of mushroom identification and the limited benchmark size, the current system should be interpreted as a research prototype rather than a reliable field-identification tool.

References

[1] Livsmedelsverket. Svamp – lär dig skilja på ätliga och giftiga sorter, n.d. URL https://www.livsmedelsverket.se/matvanor-halsa--miljo/kostrad/kostrad-vuxna/gronsaker-frukt-och-bar/svamp/. Accessed: 2026-04-14.

[2] Ockelbo kommun. Information om giftiga svampar / information about poisonous mushrooms, June 2024. URL https://ockelbo.se/nyheter/nyheter/2024-06-18-information-om-giftiga-svampar-information-about-poisonous-mushrooms. Accessed: 2026-04-14.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[4] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for mobilenetv3. *arXiv preprint*, 1905.02244, 2019. https://arxiv.org/abs/1905.02244.

[5] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint*, 1905.11946, 2019. https://arxiv.org/abs/1905.11946.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv preprint*, 2005.14165, 2020. https://arxiv.org/abs/2005.14165.

[8] Jae Joong Lee, M. Catherine Aime, Bartek Rajwa, and Euiwon Bae. Machine learning-based classification of mushrooms using a smartphone application. *Applied Sciences*, 12(22):11685, 2022. doi: 10.3390/app122211685.

[9] Pelle Holmberg and Hans Marklund. *Nya svampboken*. Bokförlaget Prisma, tredje reviderad upplaga edition, 2002. ISBN -91-518-4014-6.

[10] Gregor Hagedorn, Gerhard Rambold, and Stefano Martellos. Types of identification keys. In Pier Luigi Nimis and Régine Vignes Lebbe, editors, *Tools for Identifying Biodiversity: Progress and Problems. Proceedings of the International Congress, Paris, September 20–22, 2010*, pages 59–64, Trieste, 2010. EUT Edizioni Università di Trieste. ISBN 978-88-8303-295-0.

[11] OECD. Explanatory memorandum on the updated OECD definition of an AI system. Technical report, OECD Publishing, Paris, 2024. URL https://www.oecd.org/content/dam/oecd/en/publications/reports/2024/03/explanatory-memorandum-on-the-updated-oecd-definition-of-an-ai-system_3c815e51/623da898-en.pdf. Accessed 28 May 2026.

[12] IBM. What are large language models (llms)? https://www.ibm.com/think/topics/large-language-models, 2025. Accessed: 2026-06-04.

[13] IBM. What is multimodal ai? https://www.ibm.com/think/topics/multimodal-ai, 2024. Accessed 28 May 2026.

[14] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=RIu5lyNXjT.

[15] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://openreview.net/forum?id=bzs4uPLXvi.

[16] Wen Huang, Hongbin Liu, Minxin Guo, and Neil Gong. Visual hallucinations of multi-modal large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9614–9631. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.573. URL https://aclanthology.org/2024.findings-acl.573/.

[17] Dave Bergmann. What is image segmentation? https://www.ibm.com/think/topics/image-segmentation, 2024. IBM Think. Accessed 22 May 2026.

[18] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4 edition, 2018. ISBN - 9780133356724.

[19] OpenCV. Color space conversions. https://docs.opencv.org/4.x/d8/d01/group__imgproc__color__conversions.html, 2026. Accessed: 2026-06-04.

[20] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, Berkeley, CA, 1967. University of California Press. URL https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992.

[21] M. Emre Celebi. Improving the performance of k-means for color quantization. *Image and Vision Computing*, 29(4):260–271, 2011.

[22] OpenCV. Structural analysis and shape descriptors. https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html, 2026. Accessed: 2026-06-04.

[23] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986.

[24] Richard O. Duda and Peter E. Hart. Use of the hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972. doi: $10.1145/361237.361242$.

[25] OpenCV. cv::simpleblobdetector class reference. https://docs.opencv.org/4.x/d0/d7a/classcv_1_1SimpleBlobDetector.html, 2026. Accessed: 2026-06-04.

[26] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[27] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, pages 31210–31227, 2023.

[28] Pelle Holmberg and Görel Kristina Näslund. *Vår svampbok*. Rabén & Sjögren, Stockholm, 1982. ISBN -91-29-55614-7.

[29] Roboflow. Roboflow: Computer vision tools for developers and enterprises. https://roboflow.com/, 2026. Accessed: 2026-06-04.

[30] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. URL https://arxiv.org/abs/2408.00714.

[31] Google. Google colab faq. https://research.google.com/colaboratory/faq.html, 2026. Accessed: 2026-06-04.

[32] OpenCV. Opencv documentation. https://docs.opencv.org/4.x/, 2026. Accessed: 2026-06-04.

A Appendix A: Project Artefact Overview

The thesis is based on the repository `AI-Based-Mushroom-Identification-Using-Image-Recognition-and-Trait-Based-Cl`
The original project contains application code, training scripts, data files, model checkpoints, and benchmark outputs.
The benchmark reported in this thesis uses the Python identification components directly; the Flutter client and Java proxy backend are part of the broader software artefact but are not evaluated in Benchmark 5.

A.1 Benchmark-Relevant Source Code

The following repository components are directly relevant to the benchmarked pipeline:

- `benchmarks/`: comparative benchmark implementation. The main entry point is `benchmarks/run_comparative.py`. The `benchmarks/runners/` directory contains the standalone method runners for the CNN, LLM baselines, key traversal, trait database, and unified systems.
- `models/cnn_classifier.py`: EfficientNet-B3 species classifier inference code. The classifier loads the trained checkpoint from `artifacts/cnn_weights.pt`.
- `scripts/train_cnn.py` and `config/image_model_config.py`: CNN training entry point and image model configuration. These files define the EfficientNet-B3 transfer learning setup used to train the species classifier.
- `models/mushroom_segmenter.py`: YOLOv8 segmentation wrapper used to detect mushroom part masks. The active checkpoint is loaded from `data/Yolov8/best.pt`.
- `models/yolo_part_masks.py`: converts YOLOv8 segmentation instances into part masks for cap, stem, underside, and whole-body union masks.
- `models/visual_trait_extractor.py`: part-aware visual trait extraction. It computes colour, shape, surface, stem, underside, puffball, and morphology-case traits from the segmentation masks.
- `models/key_tree_traversal.py`: deterministic identification-key traversal engine used by the standalone key benchmark methods.
- `models/trait_database_comparator.py`: trait-database matching module. It compares visible traits against species profiles and reports a ranked candidate list.
- `models/unified_pipeline.py` and `models/llm_classifier.py`: unified multimodal pipeline and LLM synthesis logic used by B1 and B2.
- `benchmarks/species_trait_oracle.py`: oracle-trait provider used in A2 and B2 to replace automatically extracted traits with ground-truth trait dictionaries.

A.2 Data and Knowledge Files

The benchmark uses several structured data files:

- `benchmarks/evaluation_manifest_v2.csv`: Benchmark 5 manifest containing specimen identifiers, above-view and below-view image paths, ground-truth species identifiers, scenario labels, and lookalike metadata.
- `data/raw/Benchmark/`: held-out benchmark image pairs used by Benchmark 5.
- `data/raw/images/`: species-labelled training images used for CNN transfer learning. The CNN is trained from image labels, not from annotated morphological traits.
- `data/raw/key.xml`: XML-encoded Swedish identification key used by the programmatic key traversal engine and by the LLM key-navigation prompt.
- `data/raw/species_traits.xml`: structured morphological trait database used by the database comparator and oracle-trait provider.
- `data/raw/species.csv`: species metadata, including species identifiers, common names, scientific names, edibility, and toxicity categories.
- `data/raw/lookalikes.csv`: documented lookalike relationships and safety-relevant distinguishing information.

A.3 Model Checkpoints and Training Artefacts

The benchmark uses fixed trained checkpoints rather than retraining models during evaluation:

- `artifacts/cnn_weights.pt`: trained EfficientNet-B3 species classifier checkpoint.
- `data/Yolov8/best.pt`: trained YOLOv8-nano segmentation checkpoint for cap, stem, and underside detection.
- `artifacts/cnn_training_history.json`: saved CNN training history, when present.
- `Google colab/Yolov8_training.ipynb`: Colab notebook used for YOLOv8-nano segmentation training.
- `Google colab/Mushroom_Benchmark_Colab.ipynb`: Colab notebook used for benchmark execution with the trained checkpoints.
- `Google colab/`: Colab material for the training and benchmark workflow. The concrete Colab procedure is described in Appendix B.

A.4 Benchmark Outputs

Benchmark 5 writes its result artefacts to:

`benchmarks/Results/5/extract/`

The main files are:

- `report.json`: structured benchmark report with per-specimen subsystem metadata.
- `report.csv`: flat per-specimen report used to create the result tables.
- `report.md`: Markdown summary of the benchmark run.
- `checkpoint.json`: saved runner outputs used to inspect method-specific metadata such as key traversal paths and visible traits.

A.5 Application Components Outside the Benchmark Scope

The repository also contains application-facing components:

- `api/`: FastAPI backend routes for the identification pipeline.
- `java-backend/`: Spring Boot proxy backend.
- `mushroom_id_app/`: Flutter client application.

These components are part of the implemented software prototype, but Benchmark 5 evaluates the Python identification pipeline directly in order to compare standalone methods and unified synthesis under controlled input conditions.

B Appendix B: Benchmark Model Training and Execution in Google Colab

The benchmarked system uses two trained vision models: an EfficientNet-B3 species classifier and a YOLOv8-nano segmentation model for mushroom part detection. Both models were trained or fine-tuned using Google Colab GPU runtimes. The benchmark itself then loads the saved checkpoints from the project repository. The project includes two Colab notebooks for this workflow: `Yolov8_training.ipynb` for segmentation training and `Mushroom_Benchmark_Colab.ipynb` for benchmark execution.

B.1 Colab Runtime

Training and benchmark execution were performed in Google Colab using a GPU runtime. The practical procedure was to upload the prepared zip files directly into the Colab runtime, install the required Python dependencies, run the training script or notebook, and then download the trained checkpoint or benchmark artefacts back to the local project. This was used because local training was slower and less reliable for the larger image and segmentation workloads.

B.2 CNN Species Classifier

The CNN model was trained as a supervised species classifier using transfer learning. The base model was `efficientnet_b3` with ImageNet-pretrained weights. Training images were organised by species label under `data/raw/images/`. The model was trained to predict one of the 16 mushroom species classes used by the benchmark. No annotated morphological traits were provided to the CNN during training; the supervision signal was only the species label associated with each image.

The Colab procedure for the CNN consisted of the following steps:

1. Zip the training image directory locally:

   ```
   zip -r cnn-images.zip data/raw/images/
   ```

2. Upload the zip file directly into the Colab runtime and extract it.
3. Extract the project data and install dependencies:

   ```
   !pip install -q timm torch torchvision pillow
   ```

4. Run the CNN training script:

   ```
   !python scripts/train_cnn.py --epochs 30 --batch-size 16 --lr 3e-4
   ```

5. Download the best checkpoint from the Colab runtime and place it in the local project as:

   ```
   artifacts/cnn_weights.pt
   ```

The training script uses a two-phase transfer-learning procedure. First, only the classification head is trained while the ImageNet backbone is frozen. The full network is then fine-tuned with a lower learning rate. The model selected for the benchmark is the checkpoint with the best validation accuracy.

B.3 YOLOv8 Part-Segmentation Model

The segmentation model was trained to detect anatomical mushroom parts rather than species. The model was based on `yolov8n-seg`, the nano segmentation variant of YOLOv8. The training data were annotated in Roboflow with masks for three part classes: cap, stem, and underside. Null-class images were also included so that the model was exposed to images where no mushroom part should be detected.

The YOLOv8 segmentation model was trained using the project notebook `Yolov8_training.ipynb`. The notebook expects the Roboflow YOLOv8 export to be uploaded as a zip file in the Colab file browser and then extracts it into `/content/mushroom_dataset`. The procedure consisted of the following steps:

1. Export the annotated segmentation dataset from Roboflow in YOLOv8 format.

2. Upload the zip file directly into the Colab runtime. The notebook then extracts the uploaded file:

```
import zipfile
import os

zip_files = [f for f in os.listdir('/content/') if f.endswith('.zip')]
zip_path = os.path.join('/content/', zip_files[0])
zipfile.ZipFile(zip_path, 'r').extractall('/content/mushroom_dataset')
```

3. Fix the dataset paths in `data.yaml` so that they point to the extracted Colab directories. The notebook also contains a split step that can create training, validation, and test folders inside `/content/mushroom_dataset` when needed.

4. Install Ultralytics and train the segmentation model:

```
!pip install -q ultralytics

from ultralytics import YOLO

model = YOLO('yolov8n-seg.pt')
model.train(
    data='/content/mushroom_dataset/data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20
)
```

5. Download the best checkpoint from the Colab run:

```
from google.colab import files

!find /content/ -name "best.pt" -type f 2>/dev/null
# Download the best.pt path reported by the find command.
files.download('/content/runs/segment/<run-name>/weights/best.pt')
```

6. Place the downloaded checkpoint in the local project directory used by the segmentation pipeline:

```
data/Yolov8/best.pt
```

The trained YOLOv8 model is used only for part segmentation. It outputs mushroom-part masks, part labels, and detection confidence values. These masks are then passed to the classical visual trait extractor, which computes colour, shape, texture, stem, and underside traits inside the detected regions.

B.4 Benchmark Use of the Checkpoints

After training, the saved CNN and YOLOv8 checkpoints were copied back into the project repository. Benchmark 5 used these fixed checkpoints when evaluating the standalone and unified methods. The CNN checkpoint provided the species classification signal, while the YOLOv8 checkpoint provided the cap, stem, and underside masks used by the trait extractor. The benchmark did not retrain models during evaluation; it only loaded the trained checkpoints and executed the identification pipeline on the held-out benchmark specimens.

B.5 Running the Benchmark in Google Colab

The comparative benchmark can also be run in Google Colab. In this case the Colab runtime is used as the execution environment for the Python benchmark pipeline. The trained CNN and YOLOv8 checkpoints must already be present in the project archive before running the benchmark. The benchmark workflow is based on the project notebook `Mushroom_Benchmark_Colab.ipynb`.

The Colab benchmark procedure was:

1. Create a project archive containing the benchmark code, benchmark images, configuration files, and trained checkpoints:

```
zip -r mushroom-benchmark.zip . \
  -x ".venv/*" \
  -x ".git/*" \
  -x "__pycache__/*" \
  -x "*.pyc"
```

2. Upload the archive directly into the Colab runtime. The notebook uses `files.upload()` and renames the uploaded zip to `/content/mushroom-benchmark.zip`:

```
from google.colab import files
uploaded = files.upload()
```

3. Extract the project archive and verify that the main benchmark files and model checkpoints are present:

```
import shutil
from pathlib import Path

zip_path = Path('/content/mushroom-benchmark.zip')
project_dir = Path('/content/mushroom-project')
shutil.unpack_archive(str(zip_path), str(project_dir))
```

4. Install the Python dependencies needed by the benchmark:

```
%cd /content/mushroom-project
!pip install -q \
    pandas numpy scikit-learn \
    torch torchvision timm \
    Pillow opencv-python scikit-image \
    h5py tqdm \
    fastapi uvicorn python-multipart \
    pytest pytest-cov \
    python-dotenv pyyaml matplotlib seaborn \
    ultralytics
```

5. Start the local Ollama backend and make the selected multimodal model available to the benchmark. In the notebook, Ollama is installed, served locally, and the Gemma 3 model used by the benchmark is pulled:

```
!apt-get update -qq && apt-get install -y -qq zstd
!curl -fsSL https://ollama.com/install.sh | sh

import subprocess
import os

env = os.environ.copy()
env["OLLAMA_HOST"] = "0.0.0.0:11434"
subprocess.Popen(["ollama", "serve"], env=env)

!ollama pull gemma3:12b
```

6. Run the comparative benchmark:

```
import os

os.environ["OLLAMA_TIMEOUT"] = "1200"
os.environ["OLLAMA_NUM_PREDICT"] = "4096"
os.environ["OLLAMA_NUM_CTX"] = "8192"
os.environ["OLLAMA_BASE_URL"] = "http://localhost:11434"
```

```
output_dir = "artifacts/benchmarks/colab_run"
os.makedirs(output_dir, exist_ok=True)

!python3 -m benchmarks.run_comparative \
  --manifest benchmarks/evaluation_manifest_v2.csv \
  --output-dir {output_dir} \
  --variants all
```

7. Download the generated benchmark artefacts:

```
from google.colab import files
import shutil

shutil.make_archive("/content/benchmark_results", "zip", output_dir)
files.download("/content/benchmark_results.zip")
```

The benchmark outputs one structured JSON report, one CSV report, and one Markdown summary. The JSON file preserves detailed per-specimen metadata, including subsystem predictions, confidence values, fallback status, and agreement labels. The CSV file provides a flat per-specimen summary that is used for the result tables in this thesis.

C Benchmark Per-Specimen Results

Table 8 lists the 30 benchmark specimens used in Benchmark 5. The rows are specimens, not unique species: some species are represented by more than one image pair. Species are given as canonical species identifiers, matching the identifiers used in the benchmark manifest and result files.

Table 8: Benchmark 5 specimen list.

Specimen	Ground Truth	Scenario	Confusing Pair
AG.AU_001	AG.AU	ood	–
AG.AU_002	AG.AU	ood	–
AM.MU_004	AM.MU	confusing	NONE
AM.VI_007	AM.VI	confusing	NONE
BO.BA_010	BO.BA	confusing	BO.ED
BO.BA_011	BO.BA	confusing	BO.ED
BO.ED_013	BO.ED	confusing	BO.BA
BO.ED_014	BO.ED	confusing	BO.BA
CA.CI_016	CA.CI	confusing	HY.PS
CA.CI_017	CA.CI	confusing	HY.PS
HY.PS_019	HY.PS	confusing	CA.CI
HY.PS_020	HY.PS	confusing	CA.CI
LA.HE_022	LA.HE	standard	–
LA.HE_023	LA.HE	standard	–
LA.HE_024	LA.HE	standard	–
CO.CO_031	CO.CO	standard	–
CO.CO_032	CO.CO	standard	–
FO.BE_035	FO.BE	standard	–
FO.BE_036	FO.BE	standard	–
LY.PE_037	LY.PE	standard	–
LY.PE_038	LY.PE	standard	–
SP.CR_039	SP.CR	standard	NONE
GY.ES_041	GY.ES	ood	–
GY.ES_042	GY.ES	ood	–
GA.MA_043	GA.MA	ood	–
GA.MA_044	GA.MA	ood	–
PL.OS_045	PL.OS	ood	–
PL.OS_046	PL.OS	ood	–
CA.TU_047	CA.TU	ood	–
CA.TU_048	CA.TU	ood	–

Table 9 reports the per-specimen correctness matrix for Benchmark 5. A value of 1 means that the method's top prediction matched the ground-truth species identifier. A value of 0 means that the method returned a covered prediction, but the prediction was incorrect. A dash means that the method did not produce a covered prediction for that specimen.

Table 9: Benchmark 5 per-specimen correctness matrix.

Specimen	Truth	Scen.	CNN	A1V	A1L	A1K	A1D	A2L	A2K	A2D	B1	B2
AG.AU_001	AG.AU	ood	1	0	0	–	0	0	–	1	0	0
AG.AU_002	AG.AU	ood	1	0	0	–	0	0	–	1	1	1
AM.MU_004	AM.MU	conf.	0	1	1	–	0	1	–	1	1	1
AM.VI_007	AM.VI	conf.	1	1	0	1	0	1	1	1	1	1
BO.BA_010	BO.BA	conf.	1	0	0	–	0	1	–	0	1	1
BO.BA_011	BO.BA	conf.	1	0	0	–	0	0	–	0	0	0
BO.ED_013	BO.ED	conf.	0	0	0	–	0	1	–	0	1	1
BO.ED_014	BO.ED	conf.	1	0	0	1	0	1	1	0	1	1
CA.CI_016	CA.CI	conf.	1	1	0	–	0	1	–	1	1	1
CA.CI_017	CA.CI	conf.	0	1	1	–	0	1	–	1	0	0
HY.PS_019	HY.PS	conf.	1	0	0	1	0	0	1	0	1	1
HY.PS_020	HY.PS	conf.	1	0	0	–	0	0	–	0	1	1
LA.HE_022	LA.HE	standard	1	0	0	–	1	0	–	1	1	1
LA.HE_023	LA.HE	standard	1	0	0	–	0	0	–	1	0	1
LA.HE_024	LA.HE	standard	0	0	0	–	1	0	–	1	0	0
CO.CO_031	CO.CO	standard	0	0	0	–	0	0	–	1	0	0
CO.CO_032	CO.CO	standard	1	0	0	–	0	0	–	1	0	0
FO.BE_035	FO.BE	standard	1	1	1	–	0	1	–	1	1	1
FO.BE_036	FO.BE	standard	1	1	1	–	0	1	–	1	1	1
LY.PE_037	LY.PE	standard	1	0	1	–	0	0	–	1	1	1
LY.PE_038	LY.PE	standard	1	0	1	–	0	0	–	1	1	1
SP.CR_039	SP.CR	standard	1	0	0	–	0	0	–	1	1	1
GY.ES_041	GY.ES	ood	0	0	0	–	0	0	0	1	0	0
GY.ES_042	GY.ES	ood	0	0	0	–	0	0	0	1	0	0
GA.MA_043	GA.MA	ood	0	0	0	–	0	0	–	1	0	0
GA.MA_044	GA.MA	ood	0	0	0	–	0	0	–	1	0	0
PL.OS_045	PL.OS	ood	0	0	1	0	0	1	0	1	0	0
PL.OS_046	PL.OS	ood	0	1	1	–	0	1	–	1	0	0
CA.TU_047	CA.TU	ood	0	0	0	–	0	0	–	0	0	0
CA.TU_048	CA.TU	ood	0	0	0	–	0	0	–	0	0	0

The method abbreviations in Table 9 are: **A1V** = `a1_vision`, **A1L** = `a1_llm`, **A1K** = `a1_tree`, **A1D** = `a1_db`, **A2L** = `a2_llm`, **A2K** = `a2_tree`, and **A2D** = `a2_db`.

Table 10: Confusing lookalike results in Benchmark. Accuracy is reported as correct/total within each group.

Lookalike group	N	Method	Accuracy (correct/total)
Fly Agaric	1	CNN classifier	0/1
Fly Agaric	1	LLM, images only	1/1
Fly Agaric	1	LLM with extracted traits	1/1
Fly Agaric	1	Key traversal with extracted traits	0/1
Fly Agaric	1	Trait database with extracted traits	0/1
Fly Agaric	1	LLM with oracle traits	1/1
Fly Agaric	1	Key traversal with oracle traits	0/1
Fly Agaric	1	Trait database with oracle traits	1/1
Fly Agaric	1	Unified pipeline B1, extracted traits	1/1
Fly Agaric	1	Unified pipeline B2, oracle traits	1/1
Destroying Angel	1	CNN classifier	1/1
Destroying Angel	1	LLM, images only	1/1
Destroying Angel	1	LLM with extracted traits	0/1
Destroying Angel	1	Key traversal with extracted traits	1/1
Destroying Angel	1	Trait database with extracted traits	0/1
Destroying Angel	1	LLM with oracle traits	1/1
Destroying Angel	1	Key traversal with oracle traits	1/1
Destroying Angel	1	Trait database with oracle traits	1/1
Destroying Angel	1	Unified pipeline B1, extracted traits	1/1
Destroying Angel	1	Unified pipeline B2, oracle traits	1/1
Bay Bolete–Porcini	4	CNN classifier	3/4
Bay Bolete–Porcini	4	LLM, images only	0/4
Bay Bolete–Porcini	4	LLM with extracted traits	0/4
Bay Bolete–Porcini	4	Key traversal with extracted traits	1/4
Bay Bolete–Porcini	4	Trait database with extracted traits	0/4
Bay Bolete–Porcini	4	LLM with oracle traits	3/4
Bay Bolete–Porcini	4	Key traversal with oracle traits	1/4
Bay Bolete–Porcini	4	Trait database with oracle traits	0/4
Bay Bolete–Porcini	4	Unified pipeline B1, extracted traits	3/4
Bay Bolete–Porcini	4	Unified pipeline B2, oracle traits	3/4
Chanterelle–False Chanterelle	4	CNN classifier	3/4
Chanterelle–False Chanterelle	4	LLM, images only	2/4
Chanterelle–False Chanterelle	4	LLM with extracted traits	1/4
Chanterelle–False Chanterelle	4	Key traversal with extracted traits	1/4
Chanterelle–False Chanterelle	4	Trait database with extracted traits	0/4
Chanterelle–False Chanterelle	4	LLM with oracle traits	2/4
Chanterelle–False Chanterelle	4	Key traversal with oracle traits	1/4
Chanterelle–False Chanterelle	4	Trait database with oracle traits	2/4
Chanterelle–False Chanterelle	4	Unified pipeline B1, extracted traits	3/4
Chanterelle–False Chanterelle	4	Unified pipeline B2, oracle traits	3/4

Table 11: Standalone identification-key conclusions in Benchmark 5.

Method	Specimen	Ground truth	Tree conclusion	Correct	Conclusion route
a1_tree	AM.VI_007	Destroying Angel	Destroying Angel	Yes	Precheck from CNN ml_hint.
a1_tree	BO.ED_014	Porcini	Porcini	Yes	XML leaf reached by automatic trait answers: pores, detachable tubes, no ring, stout netted stem.
a1_tree	HY.PS_019	False Chanterelle	False Chanterelle	Yes	Precheck from CNN ml_hint.
a1_tree	PL.OS_045	Oyster Mushroom	Black Trumpet	No	XML leaf reached by automatic trait answers: ridges or veins, black or dark-grey fruiting body.
a2_tree	AM.VI_007	Destroying Angel	Destroying Angel	Yes	Precheck from CNN ml_hint.
a2_tree	BO.ED_014	Porcini	Porcini	Yes	XML leaf reached by automatic trait answers: pores, detachable tubes, no ring, stout netted stem.
a2_tree	HY.PS_019	False Chanterelle	False Chanterelle	Yes	Precheck from CNN ml_hint.
a2_tree	GY.ES_041	False Morel	Funnel Chanterelle	No	XML leaf reached by automatic trait answers: ridges or veins, grey-brown cap.
a2_tree	GY.ES_042	False Morel	Funnel Chanterelle	No	XML leaf reached by automatic trait answers: ridges or veins, grey-brown cap.
a2_tree	PL.OS_045	Oyster Mushroom	Black Trumpet	No	XML leaf reached by automatic trait answers: ridges or veins, black or dark-grey fruiting body.